






中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生数学建模竞赛

学　　校 _____

参赛队号 _____

队员姓名 1. _____

2. _____

3. _____

中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生
数学建模竞赛

题　目：基于亲和聚簇与实测驱动搜索的多核 NPU 切图调度研究

摘　要：

多核神经网络处理器要把一张计算图切成子图分到多个核心上执行，切得碎则负载均衡但跨核搬运与等待多，合得粗则通信少但缓存吃紧。本文以 100 张计算图为对象，分析边界搬运、共享带宽、流水线并行与换出换入四种机理，把 makespan 拆成核上最重的计算负载、共享 DDR 带宽上的搬运时间与等待三项，建立三个逐层递进的切图调度模型，用区间切分从构造上保证方案合法，以闭式估价排序、以官方评估程序实测把关。

针对问题一，首先由切图的边界搬运机理写出额外搬运量的闭式，由场景 A 的 Task 规则写出等待链的最长路，建立**基于亲和聚簇的场景 A 多核切图调度模型**；然后设计**链式与锥形双聚簇算法**把数据往来密切的算子绑成簇，用**沿数据边生长的装箱算法**分核并做边界交换，再以**核心亲和拓扑序区间切分算法**落成方案，并证明其自动满足无环与次序相容约束；最后以评估程序为目标做**实测驱动的爬山搜索**。在 80 张中小规模图上与官方示例方案、表调度等四种方法对比，本文方法的 5 核平均加速比比官方示例方案高 2.7。2 至 5 核的平均加速比为 **1.763**、**2.485**、**3.111**、**3.625**，逐用例结果见附录表 A1。

针对问题二，首先分析同核子图合并为一个 Task 后同核往返与等待消失、但缓存更紧且执行次序改变的机理，在问题一的基础上把 Task 归属改为核心、等待改为跨核同步，建立**核内驻留的场景 B 多核切图调度模型**；然后沿用**链式与锥形双聚簇算法**与**实测驱动的爬山搜索**求解，并把三问的方案放到彼此的场景下交叉评估取优；最后用条件弱化检验，问题一的方案放到场景 B 下有 92.0% 的组合不变慢。2 至 5 核的平均加速比为 **1.817**、**2.605**、**3.284**、**3.845**，5 核时比场景 A 高 **6.1%**，逐用例结果见附录表 A2。

针对问题三，首先分析只读 L2 只对错开时间的重复读有效的机理，在问题二的基础上把搬运拆到 DDR 与 L2 两个互不占用的带宽池，建立**双带宽池的共享 L2 多核切图调度模型**；然后按新估价用同一套算法择优与爬山；最后固定方案扫描 L2 的容量与带宽。只读 Cache 配置下 5 核平均加速比为 **3.913**，相对无 L2 的加速比由 1 核的 **1.0086** 升到 5 核的 **1.0237**，5 核平均命中率为 0.254，逐用例结果见附录表 A3 与表 A4。命中率有一到三成，makespan 却只降 0.9% 到 2.4%，因为 DDR 搬运大部分与计算重叠；容量从 256 KB 加到 4 MB，5 核相对加速比由 1.0119 升到 1.0427。

灵敏度分析表明，DDR 带宽从 60 降到 30 字节每周期时 5 核平均加速比下降 **17.7%**，升到 120 时只上升 2.4%，加速比对 DDR 带宽敏感，对 L2 参数不敏感。

关键词：多核神经网络处理器；切图调度；亲和聚簇；爬山搜索；只读 L2

目 录

一、 问题重述 **4**
1.1 问题背景 4
1.2 问题要求 4

二、 问题分析 **5**
2.1 问题一的分析 5
2.2 问题二的分析 5
2.3 问题三的分析 5
2.4 总体思路 6

三、 模型假设 **6**

四、 符号说明 **7**

五、 模型的准备 **7**
5.1 计算图与决策变量 7
5.2 切图带来的额外搬运 8
5.3 共享带宽与流水线并行 8
5.4 缓存容量与换出换入 9
5.5 数据特征与单核基线 10

六、 问题一模型的建立与求解 **11**
6.1 问题一的思路框架 11
6.2 场景 A 的机理分析 11
6.3 基于亲和聚簇的场景 A 多核切图调度模型 12
6.4 模型的求解 13
6.4.1 链式与锥形双聚簇算法 13
6.4.2 沿数据边生长的装箱算法 13
6.4.3 核心亲和拓扑序区间切分算法 14
6.4.4 实测驱动的爬山搜索 15
6.4.5 求解步骤 15
6.5 求解结果与检验 16
6.5.1 求解结果 16
6.5.2 多算法对比 17
6.5.3 模型的检验 18
6.6 本章小结 18

七、 问题二模型的建立与求解 **18**
7.1 问题二的思路框架 18
7.2 场景 B 的机理分析 19
7.3 核内驻留的场景 B 多核切图调度模型 19
7.4 模型的求解 20
7.5 求解结果与检验 20
7.5.1 求解结果 20

7.5.2 多算法对比 21
7.5.3 模型的检验 21
7.6 本章小结 22

八、 问题三模型的建立与求解 22
8.1 问题三的思路框架 22
8.2 只读 L2 的机理分析 22
8.3 双带宽池的共享 L2 多核切图调度模型 23
8.4 模型的求解 24
8.5 求解结果与检验 24
8.5.1 求解结果 24
8.5.2 多算法对比 25
8.5.3 L2 容量与带宽的影响 26
8.5.4 模型的检验 26
8.6 本章小结 27

九、 模型的灵敏度分析 27

十、 模型的评价、改进与推广 27
10.1 模型的优点 27
10.2 模型的缺点 28
10.3 模型的改进 28
10.4 模型的推广 28

参考文献 28

附录 30

一、 问题重述

1.1 问题背景

人工智能模型的规模持续增长，单个 AI 核心受制造成本、功耗与散热约束，难以独立满足算力需求，多核神经网络处理器因此成为提升并行能力与能效的主流架构[1]。多核调度要把一张完整的计算图切分为若干子图[2]，分配到多个核心上执行，并确定每个核心上子图的执行次序。它的难点在于三方面相互牵制：切得越碎并行度越高，但跨子图的数据要经核外主存 DDR 中转，占用所有核心共享的带宽；合得越粗通信越少，但单个子图越大，核内缓存越容易不够用，核内调度被迫换出换入。

本题的平台由 N 个同构 AI 核心与共享的 DDR 构成。如图 1 所示，左侧为 N 个核心经同一条通道访问 DDR，通道宽度即 DDR 总带宽 B；右侧放大了一个核心的内部，包括 Cube 矩阵计算单元、Vector 向量计算单元、私有的 L1 与 UB 两块缓存，以及负责搬入与搬出的 MTE2、MTE3 两条搬运流水线。

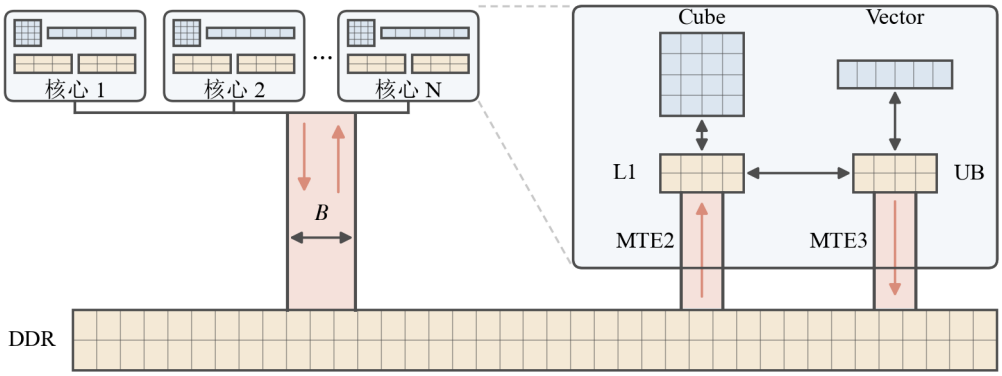


图 1 多核 NPU 平台结构与单个核心的内部构成

待调度的对象是一张有向无环计算图，由张量节点与操作节点交替相连。评估程序读入参赛方案后，按场景规则补上跨子图的边界搬运，调用给定的核内调度算法确定每个 Task 内的操作顺序并在缓存不足时插入换出换入，最后在统一时间轴上模拟所有核心在共享带宽下的并行执行，给出总体执行时间 makespan 与总额外数据搬运量。

1.2 问题要求

题目给出 100 张测试计算图与固定配置[3]：L1 容量 524288 字节，UB 容量 131072 字节，DDR 总带宽 60 字节每周期。方案只由两部分组成，一是每个非搬运操作到子图编号的映射，二是每个核心上子图编号的执行顺序。要求如下。

问题一，场景 A 下的多核切图与调度。每个子图单独构成一个 Task，所有跨子图数据都经 DDR 中转，同核前一 Task 完成后至少等待 100 周期，跨核前驱 Task 完成后至少等待 1000 周期。以缩短 makespan 为主要目标、兼顾总额外数据搬运量，在 2 到 5 核下给出方案，正文给出 1 到 5 核的平均加速比折线图，附录逐用例列出 makespan 与总额外数据搬运量。

问题二，场景 B 下的多核切图与调度。每个核心上的全部子图合并为一个 Task，同核子图可直接复用 L1 与 UB 中的数据，跨核依赖经 COPY_OUT、COPY_IN 与 500 周期同步延迟完成。要求在严格满足缓存容量约束的前提下进一步缩短 makespan、减少额外搬运，结果要求与问题一相同。

问题三，共享 L2 资源下的多核切图与调度。在场景 B 的基础上引入所有核心共享的只读 L2 Cache，容量 1 MB，带宽 250 字节每周期，与 DDR 带宽互不占用。要求研究 L2 容量与带宽对切图与调度的影响，正文给出 1 到 5 核下无 L2 与只读 Cache 两种配置的对比曲线，报告相同核数下只读 Cache 相对无 L2 的加速比，附录逐用例列出两种配置的 makespan、总额外数据搬运量与 Cache 命中率。

三问的加速比均以单核核内调度在整图上的 makespan 为基准，N 核加速比定义为基准 makespan 与 N 核评估 makespan 之比。

二、 问题分析

2.1 问题一的分析

问题一要在场景 A 下给出 2 到 5 核的切图与子图调度方案，使 makespan 尽量小。

首先，从 100 张计算图的结构入手。统计发现多数图由上百条互不相连的支路组成，并行度上界的中位数达 43.1，远超 5 核，因此瓶颈不在并行度本身，而在切开以后付出的代价。场景 A 下一个子图就是一个 Task，跨子图的数据全部要写回 DDR 再读回，同核相邻两个子图也不例外，Task 之间还要等待 100 或 1000 周期。这决定了问题一的核心矛盾：切得越碎负载越均衡，但边界搬运与 Task 等待越多。这与经典的有向无环图多处理器调度问题同构，后者已被证明是 NP 难的[4]。

然后，把 makespan 拆成核上最重的计算负载、共享带宽上的搬运时间与 Task 等待三项，建立**基于亲和聚簇的场景 A 多核切图调度模型**。模型以子图映射、子图到核心的映射与同核顺序为决策变量，以子图依赖图无环、同核顺序与依赖相容为硬约束，以评估程序给出的 makespan 为目标，并给出一个不必运行评估程序即可计算的估价函数，用来在大量候选方案之间排序。

最后，设计求解算法。先用**链式与锥形双聚簇算法**把数据往来密切的算子并成簇，再以**沿数据边生长的装箱算法**把簇分到各核心并做一轮局部改进，然后用**核心亲和拓扑序区间切分算法**把分配结果落成方案，这一步能证明两条硬约束自动成立。由于估价看不见核内调度的换出换入，最后以官方评估程序为目标函数做**实测驱动的爬山搜索**。检验方面，所有方案均由官方评估程序给出结果，并与理论下界比较；另与官方示例方案、表调度等方法做多算法对比。

2.2 问题二的分析

问题二在问题一的基础上把场景换成 B，同核的全部子图合并为一个 Task，同核子图之间的数据可以直接留在 L1 与 UB 中。

首先，比较两种场景的差别。场景 B 下同核边界不再需要 DDR 往返，也没有 Task 切换等待，只有跨核数据要写回 DDR、经 500 周期同步后再由目标核读入，因此额外搬运的归属单位从子图变成了核心，同核子图可以随意细分。

然后，在问题一模型的基础上，把 Task 归属由子图改为核心，把 Task 等待改为跨核同步延迟，得到**核内驻留的场景 B 多核切图调度模型**。同核数据长期驻留会占用缓存，所以模型仍须计入核内缓存容量约束。

最后，沿用问题一的求解框架，把一核一子图的限制去掉，估价与爬山的目标改为场景 B 的 makespan，并以场景 A 的结果作对照，用条件弱化的方法检验两种场景的结果是否符合机理。

2.3 问题三的分析

问题三在问题二的基础上加入所有核心共享的只读 L2 Cache，并要求研究其容量与带宽的影响。

首先，分析 L2 的作用机理。L2 按逻辑张量编号缓存 COPY_IN 读过的数据，同一块数据被第二个核心读取时可以命中，命中走的是与 DDR 无关的 250 字节每周期带宽。它能缓解的只是跨核的重复读，而且几个核心要在时间上错开去读，同一时刻发出的读请求都在缓存填上之前，会全部未命中。

然后，在问题二模型的基础上，把 DDR 带宽池拆成 DDR 与 L2 两个互不占用的带宽池，重复读除第一次外计入 L2 池，得到**双带宽池的共享 L2 多核切图调度模型**。

最后，用同样的算法按新模型重新选方案，给出 1 到 5 核下两种配置的对比与加速比，再固定方案改变 L2 的容量与带宽，分析两者对结果的影响。

2.4 总体思路

三问共用一套骨架，逐问加入新的机制。如图 2 所示，第一行为三问共用的数据分析与机理准备；第二行为问题一的聚簇、装箱、合法化与实测搜索；第三行在问题一基础上改为核心级 Task；第四行再加入 L2 带宽池并分析容量与带宽的影响。

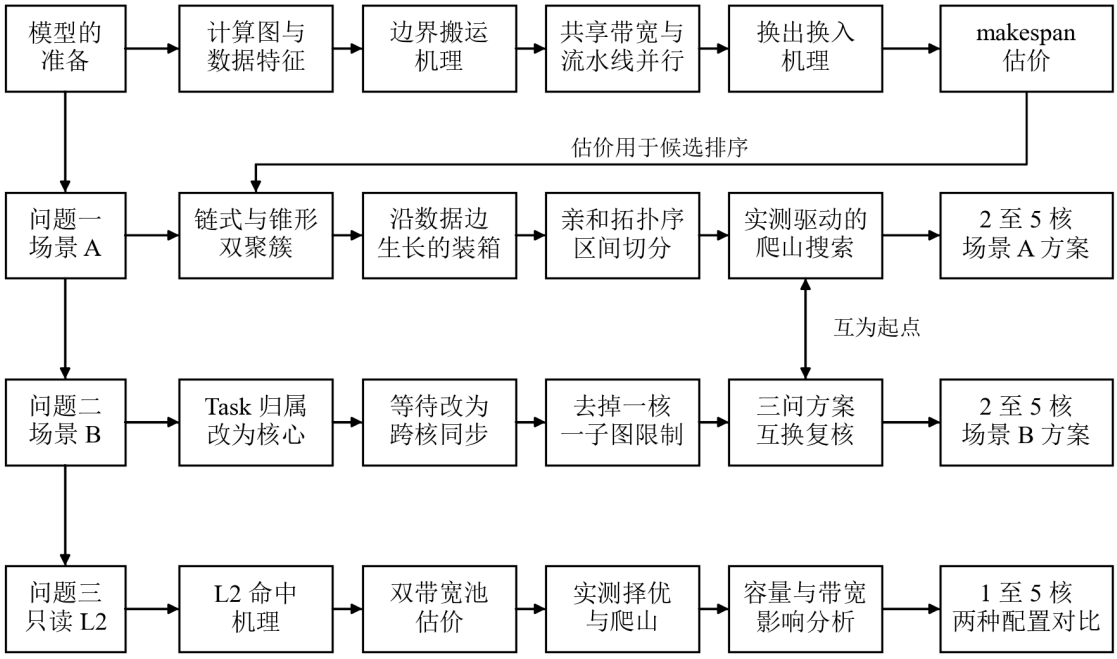


图 2　全文总体思路

三、　模型假设

(1) 计算图与配置参数如题给出，各操作的周期数、各张量的字节数在执行中不变。

(2) 核内调度算法与多核模拟执行过程完全按题目附件给出的评估程序进行，makespan 与额外搬运量以评估程序的输出为准。

(3) 同构核心之间除共享 DDR 带宽与问题三的 L2 外互不干扰，同一核心上 Cube 与 Vector 两条计算流水线可以并行。

(4) 所有在途的 DDR 搬运公平分享总带宽，问题三中 L2 命中的读取公平分享 L2 带宽，两池互不占用。

(5) 调度方案离线确定，执行过程中不因实际时序而改变子图划分与核心分配。

四、 符号说明

表 1 符号说明

符号	说明	单位
$G=(V,E)$	计算图，$V=T\cup O$ 为张量与操作节点	–
O'	核内操作集合，不含 COPY_IN 与 COPY_OUT	–
N	核心数	个
c_o, $p(o)$	操作 o 的周期数与流水线	周期
s_t	张量 t 的字节数	字节
$\sigma(o)$	操作 o 所属子图编号	–
$\pi(g)$	子图 g 所在核心	–
H_σ	子图依赖图	–
$\omega(o)$	操作 o 所属 Task，场景 A 为子图、场景 B 为核心	–
L_k	核心 k 的计算负载	周期
B, B_{L2}	DDR 总带宽与 L2 带宽	字节/周期
D_0, ΔD	原图 DDR 搬运量与总额外数据搬运量	字节
W	场景 A 的 Task 等待	周期
δ	场景 B 跨核拷贝同步延迟	周期
M_N	N 核下的 makespan	周期
S_N	N 核加速比	–
ρ_N	只读 Cache 相对无 L2 的加速比	–
h	Cache 命中率，按字节计	–

其他符号在文中出现时具体说明。

五、 模型的准备

三问共用同一张计算图、同一套核内调度与同一条共享带宽，本章先把这几处共用的机理与数据口径交代清楚，后面三章只写各问新增的部分。

5.1 计算图与决策变量

题目中的计算图由张量节点与操作节点交替相连。大张量先被切成分块，每个分块上的计算成为一个操作。如图 3 所示，(a) 中矩阵乘 $C=AB$ 的输出按块切开，C 的一个分块只依赖 A 的一行分块与 B 的一列分块；(b) 为这一分块在图中的样子，两块输入从 DDR 经 COPY_IN 搬进核内，MATMUL 算出结果后再由 COPY_OUT 写回 DDR。

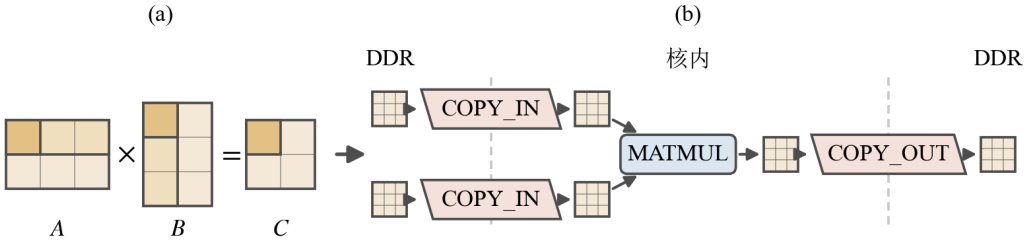


图 3 张量分块与计算图：(a) 矩阵乘的输出分块；(b) 一个分块对应的节点

记计算图为 $G=(V,E)$，$V=T\cup O$，T 为张量节点，O 为操作节点。方案只给不含 COPY_IN 与 COPY_OUT 的操作分配子图，这部分记为 O'，称为核内算子。每个 $o\in O'$ 有流水线 $p(o)\in\{\mathrm{M},\mathrm{V}\}$ 与周期数 c_o，每个张量 t 有字节数 s_t 与位置，位置为 DDR 的张量是整图的输入或输出。两个核内算子之间若有张量相连，记这条边上的数据量为

$$b_{uv}=\sum_{t:\,u\to t\to v} s_t. \tag{1}$$

方案由三组变量决定：子图映射 $\sigma: O'\to\{0,\dots,K-1\}$，子图到核心的映射 $\pi:\{0,\dots,K-1\}\to\{0,\dots,N-1\}$，以及每个核心上子图的先后次序 $\prec_k$。由 σ 可以导出子图依赖图 H_σ：若存在边 $u\to t\to v$ 且 $\sigma(u)\neq\sigma(v)$，则在 H_σ 中连一条 $\sigma(u)\to\sigma(v)$。题目的两条硬约束为 H_σ 无环，以及同一核心上的次序 $\prec_k$ 与 H_σ 相容。后文还要用到算子所属的 Task，记为 $\omega(o)$，它在场景 A 下等于 $\sigma(o)$，在场景 B 下等于 $\pi(\sigma(o))$。

5.2 切图带来的额外搬运

切图的直接代价是边界搬运。如图 4 所示，(a) 中 MUL 与 ADD 本来经一块核内张量直接相连；(b) 中两者被分进不同的 Task，评估程序在 MUL 之后插入 COPY_OUT 把这块张量写到 DDR，再在 ADD 之前插入 COPY_IN 读回。

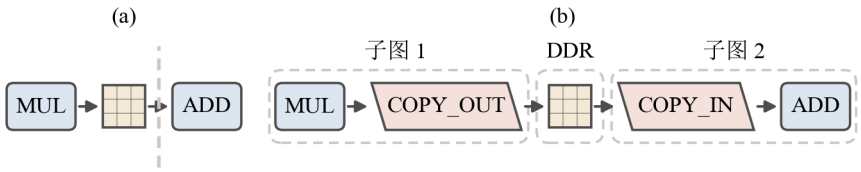


图 4　切图的边界搬运：(a) 切开之前；(b) 切开之后

据此可以写出额外搬运量。记 p_t 为张量 t 的生产者，C_t 为消费者集合，核内张量 t 的消费者落在 $R_t=\omega(C_t)\setminus\{\omega(p_t)\}$ 这些 Task 中时，要写出一次，再由每个外部 Task 各读一次；原图中的 DDR 输入本来只搬入一次，被 $|\omega(C_t)|$ 个 Task 用到时要多读 $|\omega(C_t)|-1$ 次。于是

$$\Delta D_{\mathrm{cut}}=\sum_{t\in T_{\mathrm{mid}},\,R_t\neq\emptyset} s_t\bigl(1+|R_t|\bigr)+\sum_{t\in T_{\mathrm{in}}} s_t\bigl(|\omega(C_t)|-1\bigr), \tag{2}$$

其中 T_{mid} 为核内张量，T_{in} 为 DDR 输入。DDR 输入的直接消费者是 COPY_IN 而不是核内算子，计算 C_t 时要跨过 COPY_IN 再走一步。评估程序报告的总额外搬运量还包括核内调度的换出换入 $\Delta D_{\mathrm{spill}}$，即 $\Delta D=\Delta D_{\mathrm{cut}}+\Delta D_{\mathrm{spill}}$。

5.3 共享带宽与流水线并行

所有核心经同一条通道访问 DDR。如图 5 所示，(a) 中只有一个核心在搬运，独占总带宽 B；(b) 中三个核心同时搬运，每路只分到 $B/3$。因此搬运总量不论出自哪个核心，都要在同一条通道上排队，其时间下界为 $(D_0+\Delta D)/B$，D_0 为原图本来就有的 DDR 搬运量。

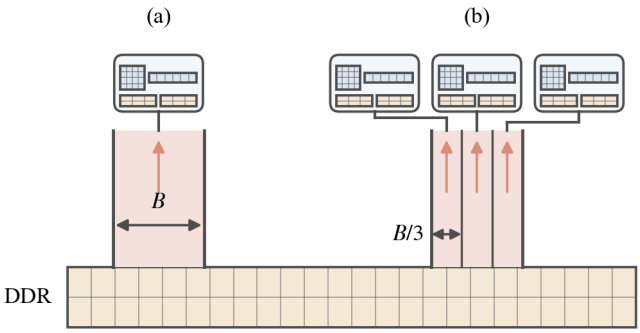


图 5　DDR 总带宽的公平分享：(a) 单核搬运；(b) 三核同时搬运

在一个核心内部，搬入、Cube 计算、Vector 计算与搬出是四条独立的流水线。如图 6 所示，第 k 块数据由 MTE2 搬入后交给 Cube，Cube 算第 k 块时 MTE2 已经在搬第 $k+1$ 块，四条流水线在时间上错开重叠。所以一个核心的计算时间由较忙的那条计算流水线决定，记核心 k 上的算子集合为 O'_k，其计算负载为

$$L_k = \max\Big(\sum_{o\in O'_k,\,p(o)=\mathrm{M}} c_o,\ \sum_{o\in O'_k,\,p(o)=\mathrm{V}} c_o\Big). \tag{3}$$

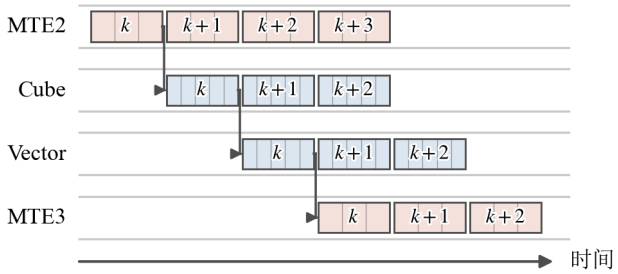


图 6　核内四条流水线的重叠执行

搬运与计算能够重叠，但总时间不会短于其中任何一项。把各核的计算负载、共享通道上的搬运时间与场景附加的等待三项相加，得到 makespan 的估价

$$\hat{M} = \max_k L_k + \frac{D_0 + \Delta D_{\mathrm{cut}}}{B} + W, \tag{4}$$

W 在场景 A 取 Task 启动等待，在场景 B 取跨核同步延迟，具体形式在各问中给出。式 (4) 把相互重叠的两项直接相加，是一个偏保守的估计，它的作用不是预测 makespan 的绝对值，而是在大量候选方案之间快速排序。

5.4　缓存容量与换出换入

核内调度要把一个 Task 的全部中间张量放进 L1 与 UB。如图 7 所示，(a) 中新张量放不下，顶出了容量线；(b) 中核内调度把一块暂时不用的张量换出到 DDR，腾出空间，等到要用时再换入[5]。换出换入也走共享带宽，并计入总额外搬运量。

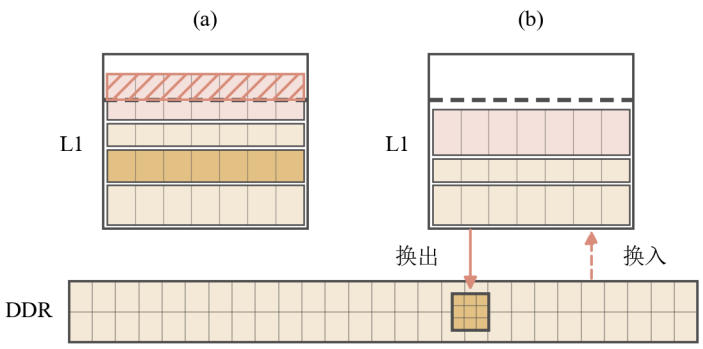


图 7　缓存不足时的换出换入：(a) 超出容量；(b) 换出与换入

这一机理对切图有两个方向相反的影响。一方面，子图越大，一个 Task 同时存活的张量越多，越容易触发换出换入；另一方面，把图切开以后每个核心只处理一部分数据，单核时的换出换入反而可能消失，使 N 核的 makespan 低于单核的 $1/N$，出现超线性加速。式 (4) 不含这一项，这正是后文要以评估程序实测来选方案的原因。

5.5　数据特征与单核基线

对 100 张计算图逐一统计结构特征，结果见表 2。其中并行度上界取两条流水线总周期与关键路径周期之比，它给出了在不计通信时最多能用上多少个核心。

表 2　100 张计算图的结构特征

特征	最小值	下四分位	中位数	最大值
核内算子数	552	1111	3721	35705
弱连通分量数	1	18	40	2666
最大分量的算子占比	0.04%	1.6%	12.2%	100%
DAG 层数	3	22	48	2440
关键路径周期	400	2501	6140	863040
并行度上界	3.42	20.2	43.1	6259
单核 makespan 与 $\max(\Sigma_{\mathrm{M}}, \Sigma_{\mathrm{V}})$ 之比	1.000	1.034	1.152	2.075
单核原图搬运 D_0/B 的时间占比	0.08%	2.3%	4.2%	19.1%

从表 2 可以看出三点。第一，多数计算图由许多互不相连的支路组成，84 张图的弱连通分量不少于 5 个，只有 16 张是单一分量；并行度上界的中位数为 43.1，只有 1 张图低于 5，因此 5 核以内的瓶颈不在并行度，而在切开以后付出的搬运与等待代价。第二，单核 makespan 与两条流水线中较忙那条的周期之比中位数为 1.152，说明单核时计算与搬运大部分是重叠的，额外时间主要来自换出换入，100 张图中有 40 张在单核时已经发生换出换入。第三，单核时 DDR 搬运时间只占 makespan 的 4.2%，但它是所有核心共享的，核数增加后计算时间按核数摊薄，搬运时间却不减反增，这一项的占比会迅速上升。这里的占比只计原图搬运 D_0，取中位数；灵敏度分析一章把换出换入与额外搬运也计入，按均值统计，单核时为 8.8%，5 核时升到 44.3%。72 张图以 Cube 计算为主，7 张图只有 Vector 计算。

三问的加速比基准 M_1 均取单核核内调度在整图上的 makespan，由题目给出的单核基线结果读出。问题三要求的 1 核结果另行在只读 Cache 配置下以整图一个子图评估得到。

六、 问题一模型的建立与求解

6.1 问题一的思路框架

问题一的求解通路如图 8 所示。先由计算图算出边上数据量与两条流水线的负载，用链式与锥形两种聚簇把算子并成簇；再以多种装箱方式把簇分到各核心，并在边界上做局部改进；然后用核心亲和拓扑序把分配落成合法方案，以式 (4) 的估价排序；最后交给官方评估程序实测择优，并在最优方案上爬山，输出 2 到 5 核的方案。

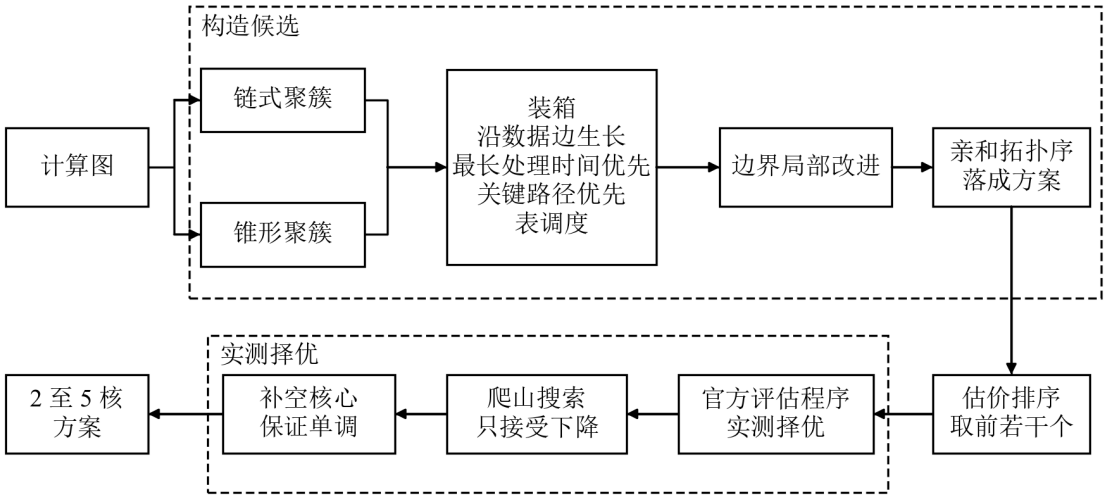


图 8 问题一的思路框架

6.2 场景 A 的机理分析

场景 A 下一个子图就是一个 Task。如图 9(a) 所示，核心 0 上有两个子图，它们之间的数据也要先写回 DDR 再读回，而且后一个 Task 必须在前一个完成后再等 $\tau_1 = 100$ 周期才能启动；核心 1 上的子图依赖核心 0 的结果，要在前驱完成后等 $\tau_2 = 1000$ 周期。图 9(b) 为问题二的场景 B，留到下一章对照。

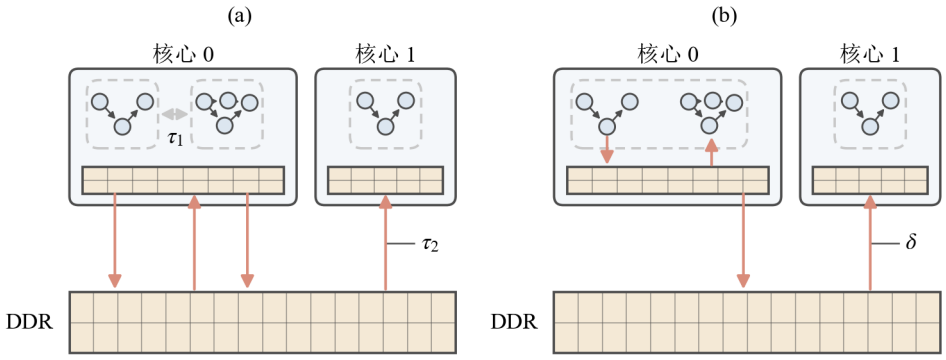


图 9 两种场景下同核与跨核的数据往来：(a) 场景 A；(b) 场景 B

由此可以得出场景 A 的两条特有机理。其一，额外搬运按子图计，式 (2) 中取 $\omega = \sigma$，同核相邻子图之间也要付出一次 DDR 往返，所以同一核心上的子图越少越好。其二，Task 等待沿依赖链累加。把子图看成节点，H_σ 的边与同核先后关系都是等待边，同核边权

为 τ_1，跨核边权为 τ_2，一个 Task 最早要在全部前驱都完成并等满相应时间后启动，等待的总代价是这张图上的最长路：

$$\psi(g) = \max_{g' \to g} \left(\psi(g') + w_{g'g} \right), \qquad w_{g'g} = \begin{cases} \tau_1, & \pi(g') = \pi(g) \\ \tau_2, & \pi(g') \neq \pi(g) \end{cases}, \qquad W_{\mathrm{A}} = \max_{g} \psi(g). \quad (5)$$

式 (5) 说明跨核依赖的代价是同核的 10 倍。一个极端的例子是每个核心只放一个子图：只要核心之间有一条数据边，后一个核心就要等前一个核心整个跑完再加 1000 周期才能开始，两个核心串成一条线，比单核还慢。所以一核一子图的落法只能用在核心之间完全没有数据边的时候。

切分粗细的权衡可以在两个核心的时间线上看清。如图 10 所示，(a) 切得碎，两个核心的计算量相同，但每段计算之后都要把数据写回 DDR，再等上一段才能启动下一个 Task，搬运与等待把时间线拉长；(b) 粗细适中，每个核心只有一两段计算，切口落在数据少的地方，makespan 最短；(c) 合成一个大子图放在核心 0 上，没有边界搬运，但缓存装不下，核内调度插入了换出换入，核心 1 全程空闲。好的方案要让切口落在数据量小的边上，同时让各核负载接近。

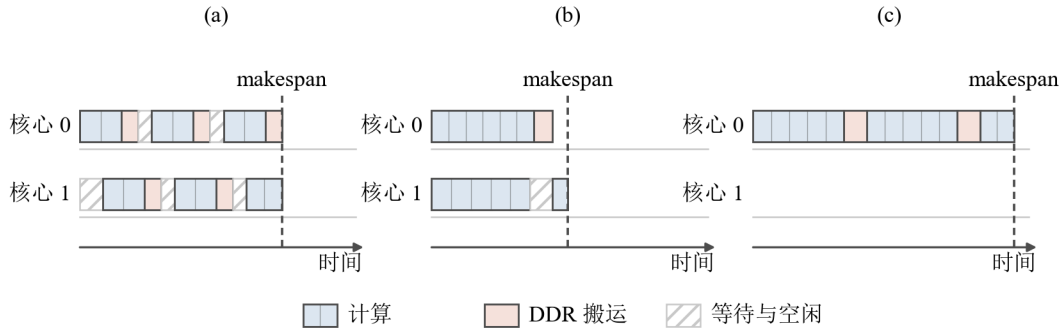


图 10　切分粗细在两个核心时间线上的表现：(a) 切得碎；(b) 适中；(c) 合得粗

6.3　基于亲和聚簇的场景 A 多核切图调度模型

决策变量的建立。决策变量为子图映射 σ、子图到核心的映射 π 与各核心上子图的次序 $\prec_k$，子图个数 K 也由 σ 决定。

目标函数的建立。题目以缩短 makespan 为主要目标，兼顾总额外搬运量，因此取两级目标：先使官方评估程序给出的场景 A 下 N 核 makespan M_N^{A} 最小，makespan 相同时再使总额外搬运量 ΔD 最小，

$$\min_{\sigma,\pi,\prec} \; \left(M_N^{\mathrm{A}}(\sigma,\pi,\prec),\; \Delta D(\sigma,\pi,\prec) \right)_{\mathrm{lex}}. \quad (6)$$

约束条件的建立。第一，每个核内算子恰好属于一个子图，每个子图恰好放在一个核心上。第二，子图依赖图 H_σ 无环[6]，等价于存在子图的一个编号 r，使每条跨子图的边都从编号小的子图指向编号大的子图。第三，同核次序与依赖相容，即若 $g \prec_k g'$，则 H_σ 中不存在从 g' 到 g 的路径。第四，记 Task ω 在核内调度下 t 时刻存活于 L1 与 UB 的张量集合为 $A_\omega^{\mathrm{L1}}(t)$ 与 $A_\omega^{\mathrm{UB}}(t)$，其字节数不得超过 524288 与 131072，不足时由核内调度插入换出换入，其字节数计入 $\Delta D_{\mathrm{spill}}$。第五，为保证加速比随核数不降，$N$ 核方案的 makespan 不得高于 $N-1$ 核方案。

综上所述，得到**基于亲和聚簇的场景 A 多核切图调度模型**：

$$
\begin{aligned}
&\min_{\sigma,\pi,\prec}\ (M_N^{\mathrm{A}},\ \Delta D)_{\mathrm{lex}}\\
&\text{s.t.}\ \begin{cases}
\sigma : O' \to \{0,\dots,K-1\}, \quad \pi : \{0,\dots,K-1\} \to \{0,\dots,N-1\}\\
\exists\, r:\ r(\sigma(u)) < r(\sigma(v)), \quad \forall\, u \to t \to v,\ \sigma(u) \neq \sigma(v)\\
g \prec_k g' \ \Rightarrow\ g' \not\rightarrow g \text{ 于 } H_\sigma, \quad \forall k\\
\sum_{x\in A_\omega^{\mathrm{LI}}(t)} s_x \le 524288, \quad \sum_{x\in A_\omega^{\mathrm{UB}}(t)} s_x \le 131072, \quad \forall \omega, t\\
\omega(o) = \sigma(o), \quad \Delta D = \Delta D_{\mathrm{cut}}(\omega) + \Delta D_{\mathrm{spill}}\\
M_N^{\mathrm{A}} \le M_{N-1}^{\mathrm{A}} \le \cdots \le M_1
\end{cases}
\end{aligned}
\tag{7}
$$

其中 ΔD_{cut} 由式 (2) 给出。M_N^{A} 与 $\Delta D_{\mathrm{spill}}$ 没有闭式，只能由评估程序模拟得到。求解时另用式 (4) 在场景 A 下的具体形式给候选方案快速排序，

$$
\hat{M}_{\mathrm{A}} = \max_k L_k + \frac{D_0 + \Delta D_{\mathrm{cut}}}{B} + W_{\mathrm{A}},
\tag{8}
$$

L_k 由式 (3) 给出，W_{A} 由式 (5) 给出。它只是排序用的估价，不属于模型的约束。

6.4 模型的求解

模型 (7) 的决策空间随算子数指数增长，最大的图有 35705 个核内算子，精确求解不可行；目标又要靠模拟才能得到，大图上单次评估耗时较长。因此采用先构造、后择优的思路：用聚簇与装箱快速构造一批结构不同的候选，用估价挑出其中较好的，再交给评估程序实测，最后在实测最优的方案上做局部搜索。

6.4.1 链式与锥形双聚簇算法

聚簇的目的是把数据往来密切的算子绑在一起，后续整簇分核，切口就不会落在数据量大的边上。计算图中常见两类结构，一类是逐元素算子串成的长链，一类是许多分块汇入同一个输出的锥体，因此借鉴线性聚簇的思想[7]，设计两种聚簇。

链式聚簇把边按数据量 b_{uv} 从大到小排序，依次做贪心匹配：若 u 还没有接出后继且 v 还没有接入前驱，就把 u、v 并入同一簇。每个算子至多接一个前驱和一个后继，所以每个簇都是图上的一条链。剩下没配上的算子再沿拓扑序接一个空闲的前驱。

锥形聚簇按逆拓扑序让每个算子并到与它数据往来最多的后继上，

$$
\mathrm{par}(o) = \arg\max_{v\in \mathrm{succ}(o)} b_{ov},
\tag{9}
$$

一个簇就是喂给同一个输出的整个锥体，分块计算的图中它正好对应一块输出分块。两种聚簇各有所长，求解时两者都用，由后面的实测来选。

6.4.2 沿数据边生长的装箱算法

记簇 c 的两条流水线周期为 ℓ_c^{M} 与 ℓ_c^{V}，以簇为节点、簇间数据量为边权得到簇图，并求出它的弱连通分量。装箱的目标负载取 $\theta = \eta \max(\Sigma_{\mathrm{M}}, \Sigma_{\mathrm{V}})/N$，$\eta \ge 1$ 为松弛系数。分量按负载从大到小依次处理：若最闲的核心装得下整个分量，就整块放入，这样不产生任何跨核数据；装不下的大分量从最重的簇起沿数据边生长，每次挑与已装部分数据往来最多的簇加入，装满 θ 就换下一个核心，切口因此自然落在数据量最少的位置。

除此之外还构造三类候选。最长处理时间优先装箱[8] 不管连通性，只把簇按负载从大到小放进最闲的核心，图很碎时配得更平。关键路径优先装箱把关键路径穿过的簇全

部放在同一个核心上，其余簇按负载配平，适用于深而窄的图，因为切口每穿过一次关键路径，路径上就多一次跨核等待。表调度[9] 按向上秩把算子逐个放到能最早完成的核心上，跨核时加上等待与搬运的代价，作为对照基线。

装箱之后借鉴 Kernighan–Lin 算法的边界交换思想[10] 做局部改进。只看边界上的簇，逐个尝试把它搬到别的核心，目标取

$$J = \max_{k} L_k + \frac{\Delta D_{\mathrm{cut}}}{B}, \tag{10}$$

J 下降就搬，重复至多四轮或不再有簇移动为止。J 的变化只涉及这个簇碰到的张量，可以增量计算，大图上只试切口最粗的 4000 个簇。

6.4.3 核心亲和拓扑序区间切分算法

装箱给出的是算子到核心的分配，还要把它落成满足约束的子图与次序。做法是按核心亲和的拓扑序排开全部算子：维护每个核心的就绪队列，当前核心还有就绪算子就接着取，取空了才换到就绪算子最早能完成的核心上。记算子在排出的序列中的位置为 $\lambda(o)$，第 i 个算子为 o_i，同核连续的一段为一个子图，子图编号即此前换核的次数，

$$\sigma(o) = \left|\{\, i < \lambda(o) : \pi(o_i) \neq \pi(o_{i+1}) \,\}\right|, \tag{11}$$

各核心按子图编号递增执行。亲和的作用是让同一核心上的算子尽量排在一起，段数少，子图就少，场景 A 下的边界搬运与等待都随之减少。

命题 1. *按式 (11) 得到的方案满足模型 (7) 的无环约束与次序相容约束。*

证明. λ 是拓扑序，任一条边 $u \to v$ 有 $\lambda(u) < \lambda(v)$。σ 沿 λ 单调不减，所以 $\sigma(u) \le \sigma(v)$，两者不等时 $\sigma(u) < \sigma(v)$。取 r 为子图编号本身，H_σ 的每条边都从小编号指向大编号，故 H_σ 无环。同一核心按编号递增执行，若 $g \prec_k g'$ 则 $g < g'$，而 H_σ 中的路径只能使编号增大，不存在从 g' 到 g 的路径，次序与依赖相容。 □

如图 11 所示，(a) 中链 a 分在核心 0 上，画成实心；链 b 分在核心 1 上，画成空心；两条跨核边为 a1 到 b2 与 b3 到 a3。亲和序先在核心 0 上排完 a1、a2，a3 要等 b3，于是换到核心 1 排完 b1 至 b3，再回到核心 0 排 a3，两处换核用虚线切开，得到 g_1、g_2、g_3 三段，所有边都从左指向右。(b) 为由此得到的子图依赖图，只有从小编号指向大编号的边，核心 0 依次执行 g_1 与 g_3，核心 1 执行 g_2。核心之间一条数据边都没有时，每个核心直接取一个子图，一核一子图只在这种情形下不会增加等待。

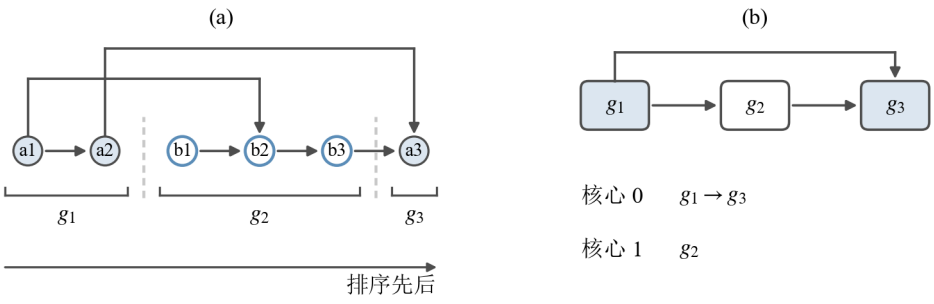


图 11 亲和拓扑序区间切分：(a) 排序与切分；(b) 子图依赖与执行次序

6.4.4　实测驱动的爬山搜索

式 (4) 看不见换出换入，也看不见带宽随时间的起伏，估价最好的方案实测未必最好。所以把估价排在前面的候选全部交给官方评估程序实测，取 makespan 最小者为起点，再以评估程序为目标函数爬山。邻域动作有三种：以 0.40 的概率把一个子图整体搬到与它数据往来最多的核心上，以 0.45 的概率把一个锥形簇或链式簇整体搬到另一个核心上，以 0.15 的概率把一个边界算子搬到它邻居所在的核心上。每个动作改变的是算子到核心的分配，再经亲和拓扑序重新落成方案，所以搜索过程中每个方案都自动满足约束。只接受使 makespan 下降的动作。

评估次数按单次评估耗时自适应确定，每组核数与问题约用 135 秒的评估时间，至多 900 次；连续一段时间没有改进就提前停止。随机种子由用例名、核数与问题号经 MD5 散列定死，结果可以复现。核内算子超过 9000 个的 20 张大图单次评估太慢，不做爬山，取两轮实测择优中较好的方案。

最后处理单调约束。N 核的候选池中始终放入把 $N-1$ 核最优方案补上一个空核心的方案，它的 makespan 与 $N-1$ 核相同，于是

$$M_N \le M_{N-1} \le \cdots \le M_1, \qquad S_N \ge S_{N-1} \ge \cdots \ge 1. \tag{12}$$

6.4.5　求解步骤

求解流程如图 12 所示，步骤如下。

Step1：读入计算图，求出核内算子集合 O'、边上数据量 b_{uv}、两条流水线的周期与原图 DDR 搬运量 D_0。

Step2：分别做链式聚簇与锥形聚簇，得到两套簇及簇图。

Step3：对每套簇用沿数据边生长的装箱，松弛系数取 1.02、1.20、1.50、2.00，再加最长处理时间优先、关键路径优先与只用 $m < N$ 个核心的装箱，各自做一次局部改进；另加表调度的分配。

Step4：对每套分配用亲和拓扑序落成方案，场景 A 下若核心之间没有数据边，另给一核一子图的落法。

Step5：按式 (4) 估价排序，取前若干个候选，连同历史轮次的方案与补空核心的 $N-1$ 核方案一起交给评估程序实测，取最优者。

Step6：以实测最优者为起点做爬山搜索，得到最终方案。

Step7：与 $N-1$ 核的方案比较，若更慢则改用补空核心的 $N-1$ 核方案；$N < 5$ 时令 N 加 1 回到 Step3，否则输出 2 到 5 核的方案。

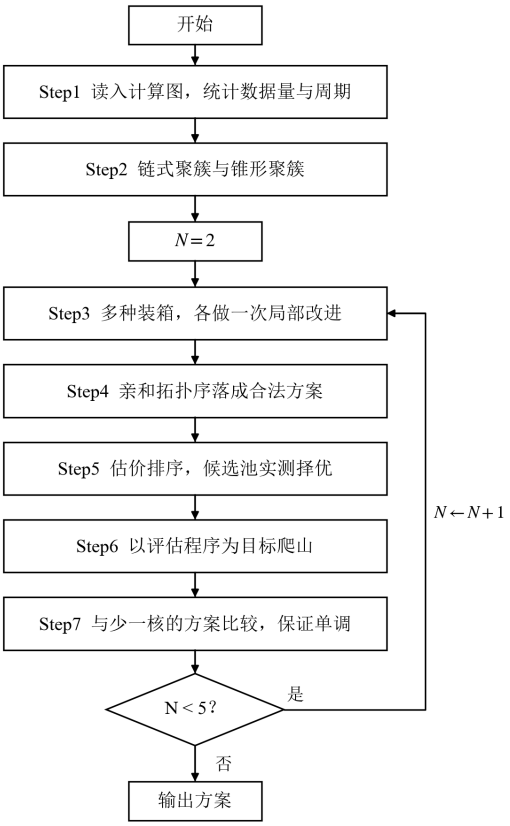


图 12　求解算法流程

6.5　求解结果与检验

6.5.1　求解结果

100 张计算图在 2 到 5 核下的 400 组方案全部通过官方评估程序的合法性检查。1 到 5 核的平均加速比与平均额外搬运量见表 3，平均加速比随核数的变化如图 13(a) 所示。逐用例的 makespan 与总额外数据搬运量见附录表 A1，完整结果见支撑材料中的 result_p1.csv。

表 3　问题一的平均加速比与平均额外搬运量

核数 N	1	2	3	4	5
平均加速比 $\bar{S}_N$	1.000	1.763	2.485	3.111	3.625
加速比中位数	1.000	1.986	2.902	3.737	4.162
平均额外搬运量/MB	12.42	12.81	15.23	15.54	16.12
平均子图数	1	2.3	26.7	28.6	31.1

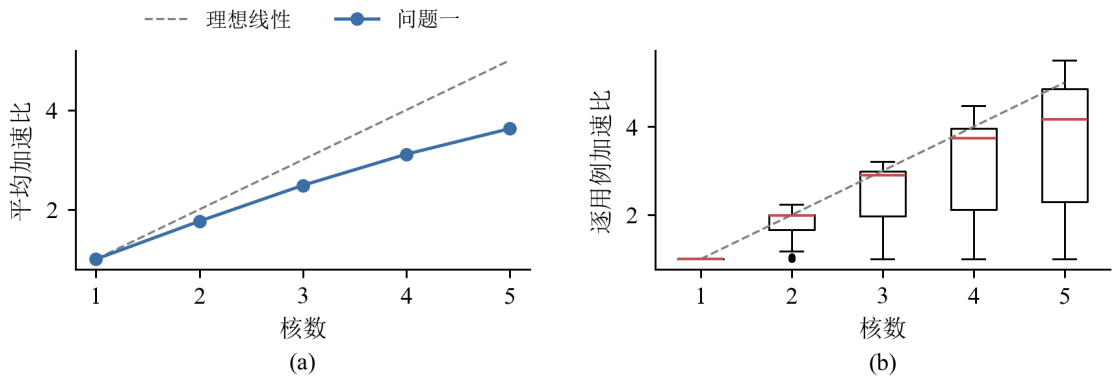


图 13　问题一的加速比：(a) 平均加速比；(b) 逐用例加速比分布

从图 13(a) 可以看出，平均加速比随核数单调上升，5 核时达到 3.625，并行效率为 72.5%。曲线在核数增加时逐渐偏离理想直线，原因是核数越多，切口越多，边界搬运越大，而 DDR 带宽是所有核心共享的，额外搬运的时间不随核数摊薄。图 13(b) 给出了逐用例的分布，5 核时有 42 个用例的加速比不低于 4.5，其中有 8 个超过 5，这些图在单核时有大量换出换入，切开以后每个核心的数据放得下，换出换入消失，出现超线性加速。分布的下端是四个用例：case_048 为 1.01，case_064 为 1.09，case_016 为 1.12，case_024 为 1.16。其中 case_048 与 case_064 只有一个连通分量，并行度上界分别只有 3.4 与 6.5，切开就要穿过依赖链。case_016 与 case_024 属于另一类结构，本文称为归约广播图：它们只用 Vector 流水线，每一层先把全部结果归约成一个值，再把这个值广播给下一层的所有分块，层与层之间是全连接的依赖，切开后每层都产生跨核依赖，场景 A 下每次还要等 1000 周期，所以只能退回单核或接近单核的方案。

表 3 中的平均子图数与额外搬运量反映了切图的代价。5 核时平均子图数为 31.1，多于核数，说明多数方案不是一核一子图，而是按跨核依赖切成若干段。平均额外搬运量从单核时只有换出换入的 12.42 MB 增加到 5 核的 16.12 MB，5 核额外搬运与原图 DDR 搬运之比的中位数为 1.7，这部分流量在共享带宽上排队，是加速比随核数增长变慢的主要原因。爬山在 50.9% 的中小规模组合上找到了更好的方案，平均使 makespan 再降 2.23%。

综上，问题一在 2、3、4、5 核下的平均加速比分别为 1.763、2.485、3.111、3.625。

6.5.2　多算法对比

为说明各个环节的作用，在 5 核下用五种方法生成场景 A 的方案，全部由官方评估程序实测。20 张大图单次评估要十几分钟，对比只在核内算子不超过 9000 个的 80 张图上进行，后文的参数实验同样取这 80 张。官方示例方案为题目附件给出的随机切图程序；表调度为经典的 HEFT 算法，逐个算子挑最早完成的核心；估价择优取式 (4) 估价最好的一个候选，不经实测；实测择优把候选池全部实测后取最优，不做爬山；本文方法在实测择优的基础上再爬山。结果见表 4。与最优的平均差距指各用例上该方法的 makespan 比五种方法中最小者高出的比例的平均值；命中最优比例指该方法取得最小 makespan 的用例比例，并列时都计入；用时为每个用例生成一套方案的平均秒数。

从表 4 可以看出，官方示例方案的平均加速比只有 0.929，随机切图使边界搬运与跨核等待成倍增加，多数用例比单核还慢。表调度只看计算时间，不看切口，5 核平均加速比为 1.212，逐个算子挑核心造成大量细碎的跨核边。估价择优把平均加速比提到 3.230，说明聚簇与装箱构造的方案本身已经较好。实测择优进一步提到 3.541，差别来自估价看不见的换出换入。爬山在此基础上再提高到 3.612。本文方法的候选池包含了

表调度与估价择优的方案，爬山又只接受下降，所以它命中最优的比例为 100% 是由构造保证的，更能说明问题的是其余方法与它的差距：实测择优平均还差 2.4%，估价择优差 19.1%。代价是用时，本文方法每个用例约需 151 秒，比估价择优多出两个数量级，但仍在分钟级，离线编译可以接受。因此最终采用实测择优加爬山的方案；对单次评估耗时较长的大图，退回实测择优即可。

表 4　问题一五种方法的对比，80 张图，$N = 5$

指标	官方示例	表调度	估价择优	实测择优	**本文方法**
平均加速比	0.929	1.212	3.230	3.541	3.612
与最优的平均差距	294.1%	263.8%	19.1%	2.4%	0.0%
命中最优比例	0%	5%	29%	49%	100%
平均用时/s	0.089	0.035	0.936	18.7	151.4

6.5.3　模型的检验

检验分两方面。一是合法性，400 组方案全部通过官方评估程序的约束检查，与命题的结论一致；加速比逐用例随核数不降，满足单调约束。二是与理论下界比较，N 核 makespan 不可能低于计算负载、关键路径与原图搬运三者中的最大者，

$$M_N \;\geq\; \mathrm{LB}_N = \max\Big(\frac{\max(\Sigma_{\mathrm{M}},\,\Sigma_{\mathrm{V}})}{N},\;\; C_{\mathrm{crit}},\;\; \frac{D_0}{B}\Big). \tag{13}$$

5 核时 makespan 与下界之比的中位数为 1.59，有 33 个用例在下界的 1.2 倍以内。这一比值与并行度上界的秩相关系数为 -0.78，16 张单一连通分量的图中位数为 3.67，其余图为 1.44。可见差距主要出在并行度低、只有一个连通分量的图上，这类图切开时必然穿过依赖链，每穿过一次就要付出边界搬运与 1000 周期的跨核等待，而下界不计任何切图代价。支路多的图则已经接近下界，说明模型在可以并行的地方把并行度用足了。

6.6　本章小结

本章建立了**基于亲和聚簇的场景 A 多核切图调度模型**，以链式与锥形双聚簇、沿数据边生长的装箱与核心亲和拓扑序区间切分构造合法候选，再以官方评估程序实测择优并爬山。问题一在 2 到 5 核下的平均加速比为 1.763、2.485、3.111、3.625，5 核时额外搬运量平均为 16.12 MB；在 80 张中小规模图上，5 核平均加速比比官方示例方案的 0.929 与表调度的 1.212 分别高出 2.7 与 2.4。

七、　问题二模型的建立与求解

7.1　问题二的思路框架

问题二在问题一的基础上把场景换成 B，求解通路如图 14 所示。聚簇、装箱与亲和拓扑序三步沿用问题一，改动集中在两处：估价中的 Task 归属由子图改为核心，Task 等待改为跨核同步延迟；落成方案时去掉一核一子图的限制，同核可以任意细分。求解顺序上先解问题二，它爬山得到的核心分配再交给问题一与问题三作起点。

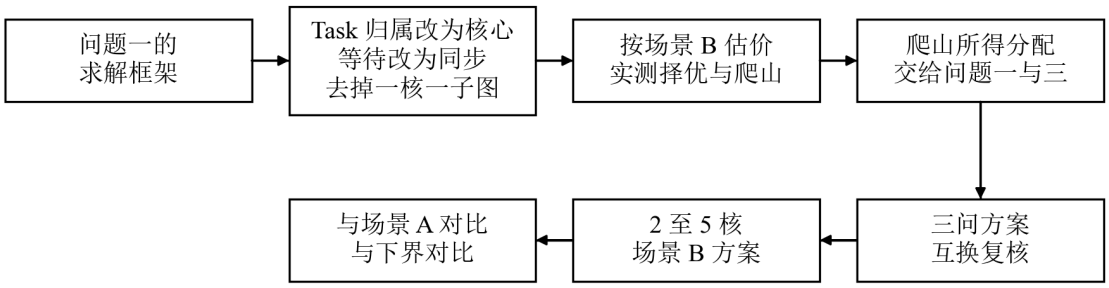


图 14　问题二的思路框架

7.2　场景 B 的机理分析

场景 B 与场景 A 的差别见图 9(b)。核心 0 上的两个子图合并为一个 Task，它们之间的数据直接留在 L1 与 UB 中，不再经 DDR 往返，也没有 100 周期的切换等待；核心 1 需要核心 0 的结果时，数据由 COPY_OUT 写回 DDR，经 $\delta = 500$ 周期同步后再由 COPY_IN 读入。

由此得到三条变化。其一，额外搬运只在跨核时发生，式 (2) 中取 $\omega(o) = \pi(\sigma(o))$，同核子图随意细分都不增加搬运，子图的划分只剩下表达执行次序的作用。其二，跨核依赖的等待从 1000 周期降为 500 周期，且同核之间不再有等待，场景 B 下

$$W_{\mathrm{B}} = \delta \cdot \max_{P} \left|\{(g', g) \in P : \ \pi(g') \neq \pi(g)\}\right|, \tag{14}$$

即沿任一条依赖链上跨核次数的最大值乘以同步延迟。求解时估价只计一次同步，取 $W_{\mathrm{B}} \approx \delta$，多次跨核的影响交给实测。其三，同核全部子图并成一个 Task 以后会带来两个副作用。一是这个 Task 在核内同时存活的张量比场景 A 下的单个子图多，缓存更容易顶满：场景 A 下每个子图结束时数据都写回 DDR，缓存随之清空，场景 B 下数据留在核内，若多块数据长时间共存就会触发换出换入。二是核内调度在整个 Task 内重新排序，不再按子图的先后执行，别的核心要等的数据可能被排到后面，下游核心的等待随之变长。因此问题二并非处处优于问题一，模型必须保留缓存容量约束。

7.3　核内驻留的场景 B 多核切图调度模型

决策变量的建立。与问题一相同，为 σ、π 与 $\prec_k$。

目标函数的建立。以场景 B 下的 N 核 makespan M_N^{B} 为主要目标，总额外搬运量为次要目标。

约束条件的建立。前三条约束与问题一相同。缓存容量约束改为按核心级 Task 计：记核心 k 的合并 Task 在核内调度下 t 时刻存活于 L1 与 UB 的张量集合为 $A_k^{\mathrm{L1}}(t)$ 与 $A_k^{\mathrm{UB}}(t)$，必须有 $\sum_{x \in A_k^{\mathrm{L1}}(t)} s_x \le 524288$ 与 $\sum_{x \in A_k^{\mathrm{UB}}(t)} s_x \le 131072$，不满足时由核内调度换出，换出换入量计入 $\Delta D_{\mathrm{spill}}$。单调约束同问题一。

综上所述，得到**核内驻留的场景 B 多核切图调度模型**：

$$
\begin{aligned}
&\min_{\sigma,\pi,\prec}\ (M_N^{\mathrm{B}},\ \Delta D)_{\mathrm{lex}} \\
&\text{s.t.}\ \begin{cases}
\sigma : O' \to \{0,\dots,K-1\}, \quad \pi : \{0,\dots,K-1\} \to \{0,\dots,N-1\} \\
\exists\, r:\ r(\sigma(u)) < r(\sigma(v)), \quad \forall\, u \to t \to v,\ \sigma(u) \neq \sigma(v) \\
g \prec_k g' \ \Rightarrow\ g' \not\rightsquigarrow g \text{ 于 } H_\sigma, \quad \forall k \\
\sum_{x \in A_k^{\mathrm{L1}}(t)} s_x \le 524288, \quad \sum_{x \in A_k^{\mathrm{UB}}(t)} s_x \le 131072, \quad \forall k, t \\
\omega(o) = \pi(\sigma(o)), \quad \Delta D = \Delta D_{\mathrm{cut}}(\omega) + \Delta D_{\mathrm{spill}} \\
M_N^{\mathrm{B}} \le M_{N-1}^{\mathrm{B}} \le \cdots \le M_1
\end{cases}
\end{aligned}
\tag{15}
$$

求解时用于排序的估价相应改为

$$
\hat{M}_{\mathrm{B}} = \max_k L_k + \frac{D_0 + \Delta D_{\mathrm{cut}}}{B} + \delta. \tag{16}
$$

7.4 模型的求解

求解框架与问题一相同，改动有四处。第一，估价按式 (16) 计算，归属单位取核心。第二，亲和拓扑序落成方案时不再尝试一核一子图，同核被跨核依赖隔开的几段各为一个子图，它们属于同一个 Task，只表达执行次序。第三，跨核代价降低以后，表调度的跨核惩罚相应取 500 周期。第四，三问按问题二、问题三、问题一的顺序求解，问题二爬山得到的核心分配交给另外两问作为额外起点，因为三者的分配结构往往相近，而问题二的方案对跨核切口最敏感。

所有三问方案生成以后，再做一步互换复核，即交叉评估：同一用例同一核数下，三问的方案格式与约束完全相同，只是评估场景不同，因此把每套方案在另外两个场景下各评一遍，若某一问的方案在另一问的场景下更快，就换过来。这一步不增加新的构造，只是让三问的方案互为候选。

7.5 求解结果与检验

7.5.1 求解结果

问题二的 400 组方案全部合法，平均加速比与平均额外搬运量见表 5，逐用例结果见附录表 A2，完整结果见支撑材料中的 result_p2.csv。

表 5 问题二的平均加速比与平均额外搬运量

核数 N	1	2	3	4	5
平均加速比 $\bar{S}_N$	1.000	1.817	2.605	3.284	3.845
加速比中位数	1.000	1.993	2.928	3.773	4.288
平均额外搬运量/MB	12.42	12.74	13.47	13.81	14.34
平均子图数	1	7.2	28.2	30.4	37.3

两种场景的对比如图 15 所示，(a) 为两种场景的平均加速比，(b) 为 5 核时逐用例的场景 A 与场景 B 的 makespan 之比。

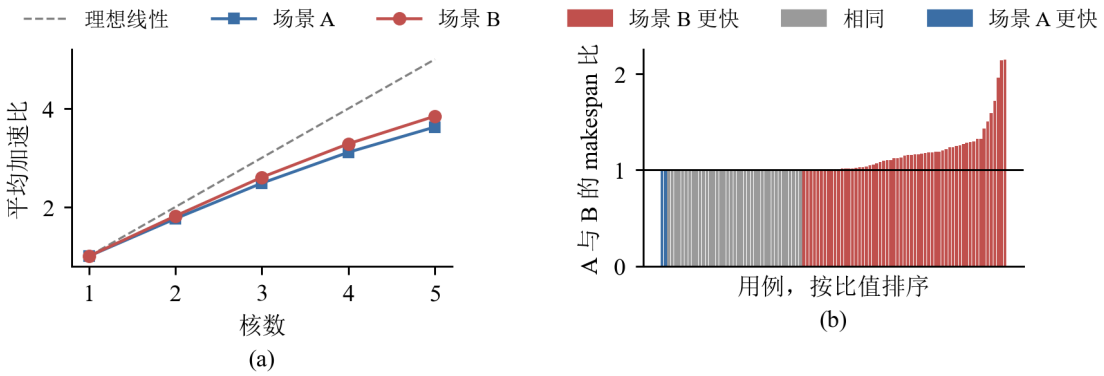


图 15　问题二与问题一的对比：(a) 平均加速比；(b) 5 核时逐用例 makespan 之比

从图 15(a) 可以看出，场景 B 的平均加速比在每个核数下都高于场景 A，5 核时为 3.845，比场景 A 高 6.1%，差距随核数增大而扩大，因为核数越多跨子图的边越多，场景 B 省下的同核往返与等待也越多。5 核时平均子图数为 37.3，比场景 A 的 31.1 多，因为同核细分在场景 B 下没有代价，亲和拓扑序会按跨核依赖把同核算子分成许多段。图 15(b) 中 100 个用例里有 59 个场景 B 更快，39 个相同，2 个场景 A 更快。比值最大的是归约广播图，它们在场景 A 下每层都要付 1000 周期的跨核等待，场景 B 下同核的层与层之间可以直接衔接，case_016、case_024 与 case_051 三张图的 5 核平均加速比由 1.32 提到 2.52。场景 A 更快的少数用例对应机理分析中指出的两个副作用。5 核时场景 B 的平均额外搬运量为 14.34 MB，比场景 A 少 11.0%，与同核往返的消失一致。

综上，问题二在 2、3、4、5 核下的平均加速比分别为 1.817、2.605、3.284、3.845。

7.5.2　多算法对比

与问题一相同，在 5 核下比较五种方法在场景 B 下的实测结果，见表 6。

表 6　问题二五种方法的对比，80 张图，$N = 5$

指标	官方示例	表调度	估价择优	实测择优	**本文方法**
平均加速比	1.199	1.517	3.139	3.701	3.847
与最优的平均差距	228.4%	202.2%	36.3%	4.8%	0.0%
命中最优比例	0%	5%	19%	21%	100%
平均用时/s	0.089	0.036	0.677	19.5	173.2

从表 6 可以看出，各方法的排序与问题一一致。场景 B 下跨核代价较低，表调度的表现比问题一好，平均加速比为 1.517，但仍不及聚簇装箱的估价择优 3.139。实测择优把平均加速比提到 3.701，比问题一中的提升更大，说明场景 B 下估价与实测的偏差更大，原因正是估价看不见合并 Task 后的换出换入与执行次序；爬山再提高到 3.847。本文方法每个用例约需 173 秒，仍在分钟级。实测择优与估价择优相比平均差距分别为 4.8% 与 36.3%，所以在场景 B 下实测环节不可省略，最终采用实测择优加爬山。

7.5.3　模型的检验

采用条件弱化的方法检验。同一套方案放到两个场景下，场景 B 只是去掉了同核子图之间的 DDR 往返与 100 周期等待，跨核等待也从 1000 周期降到 500 周期，所以按模型推断场景 B 的结果应当基本不劣于场景 A，例外只能来自机理分析中的两个副作用。

把问题一的 400 组方案直接放到场景 B 下评估，92.0% 的组合满足 $M_N^{\mathrm{B}} \le M_N^{\mathrm{A}}$。变慢的 32 个组合中只有 22% 伴随额外搬运增加，其余组合的搬运不增反减，可以推断它们变慢主要来自执行次序的改变，而不是换出换入。互换复核以后，最终结果中场景 B 不劣于场景 A 的组合占 98.0%。与式 (13) 的下界相比，5 核时 makespan 与下界之比的中位数为 1.48，比场景 A 的 1.59 更接近下界。

7.6 本章小结

本章在问题一的基础上把 Task 归属改为核心、Task 等待改为跨核同步，建立了**核内驻留的场景 B 多核切图调度模型**，并保留缓存容量约束。问题二在 2 到 5 核下的平均加速比为 1.817、2.605、3.284、3.845，5 核时比场景 A 高 6.1%。

八、 问题三模型的建立与求解

8.1 问题三的思路框架

问题三在问题二的基础上加入共享的只读 L2 Cache，求解通路如图 16 所示。先分析 L2 在什么条件下能够命中，据此把搬运拆到 DDR 与 L2 两个带宽池中，得到新的估价；再沿用问题二的构造与搜索框架按新估价选方案，给出 1 到 5 核下无 L2 与只读 Cache 两种配置的结果；最后固定方案，改变 L2 的容量与带宽，分析两者的影响。

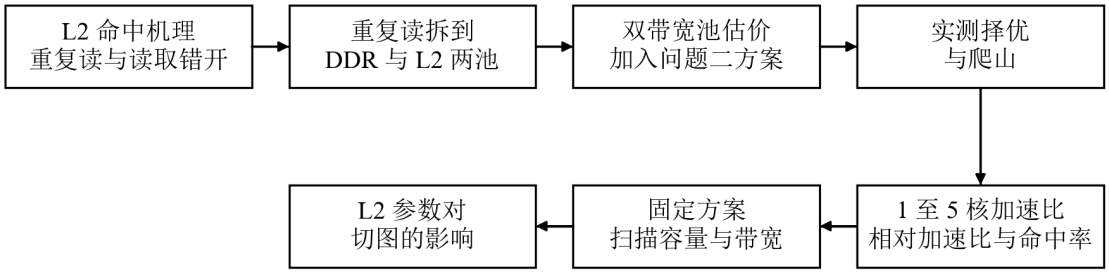


图 16 问题三的思路框架

8.2 只读 L2 的机理分析

L2 以逻辑张量编号为键，缓存 COPY_IN 从 DDR 读过的数据，容量满时按先进先出淘汰[11]，命中的读取走 L2 自己的 $B_{\mathrm{L2}} = 250$ 字节每周期带宽，与 DDR 互不占用。如图 17 所示，(a) 为无 L2 时三个核心都要读同一块数据，三路读取挤在 DDR 通道上，各分到三分之一带宽；(b) 为有 L2 时第一个核心的读取未命中，从 DDR 读入并填进 L2，另两个核心随后从 L2 命中，走的是右侧那条独立的带宽。

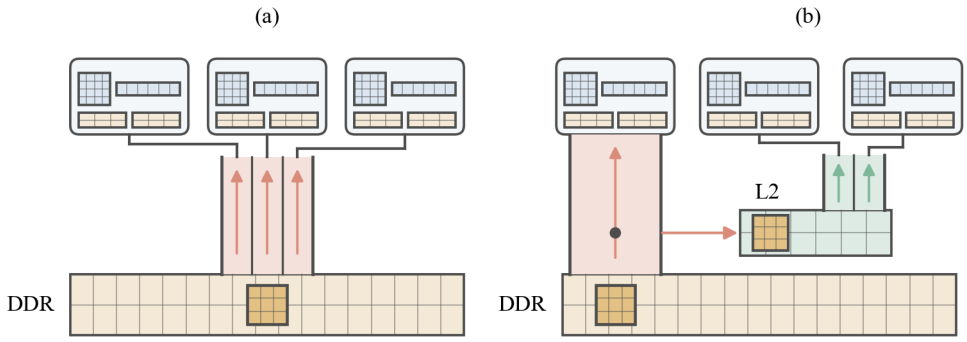


图 17　只读 L2 对重复读取的分流：(a) 无 L2；(b) 有只读 L2

由此可以看出 L2 起作用需要同时满足三个条件。第一，同一块数据要被读取两次以上，也就是 DDR 输入被多个 Task 共用，或者一块中间结果被广播给多个核心，单纯的一读一写不会命中。第二，读取要在时间上错开，几个核心在同一时刻发出的读请求都在缓存填上之前，会全部未命中。第三，数据块不能超过容量，超过 1 MB 的张量无法放入，而且在两次读取之间写入的其他数据不能把它挤出去。命中率定义为按字节计的命中比例，

$$h = \frac{D_{\text{hit}}}{D_{\text{hit}} + D_{\text{miss}}}, \tag{17}$$

只读 Cache 相对无 L2 的加速比定义为同一核数下两种配置的 makespan 之比，$\rho_N = M_N^{\text{noL2}} / M_N^{\text{Cache}}$。

L2 减少的只是 DDR 通道上的排队时间，而第五章已经指出，DDR 搬运在单核时与计算大部分重叠。只有当 DDR 通道成为瓶颈时，L2 才能换来 makespan 的明显下降，这是后面结果的关键。

8.3　双带宽池的共享 L2 多核切图调度模型

决策变量的建立。与问题二相同。

目标函数的建立。以只读 Cache 配置下的 N 核 makespan M_N^{C} 为主要目标，总额外搬运量为次要目标。

约束条件的建立。问题二的全部约束保留。新增的是搬运量在两个带宽池之间的划分。对 DDR 输入 t，若被 $n_t = |\omega(C_t)|$ 个 Task 读取且 $s_t \le \text{Cap}$，第一次读取走 DDR，其余 $n_t - 1$ 次可以从 L2 命中；对被广播到 $|R_t| \ge 2$ 个外部核心的中间张量，写出一次与第一次读回走 DDR，其余 $|R_t| - 1$ 次读回可以命中。于是

$$\begin{aligned} \Delta D_{\text{L2}} &= \sum_{t \in T_{\text{in}},\ s_t \le \text{Cap}} s_t\,(n_t - 1) + \sum_{t \in T_{\text{mid}},\ s_t \le \text{Cap},\ |R_t| \ge 2} s_t\,(|R_t| - 1), \\ \Delta D_{\text{DDR}} &= \Delta D_{\text{cut}} - \Delta D_{\text{L2}}. \end{aligned} \tag{18}$$

两个带宽池并行工作，搬运时间取两者的较大者。

综上所述，得到**双带宽池的共享 L2 多核切图调度模型**：

$$
\begin{aligned}
&\min_{\sigma,\pi,\prec}\ (M_N^{\mathrm{C}},\ \Delta D)_{\mathrm{lex}}\\
&\text{s.t.}\ \begin{cases}
\sigma : O' \to \{0,\dots,K-1\}, \quad \pi : \{0,\dots,K-1\} \to \{0,\dots,N-1\}\\
\exists\, r:\ r(\sigma(u)) < r(\sigma(v)), \quad \forall\, u \to t \to v,\ \sigma(u) \neq \sigma(v)\\
g \prec_k g' \ \Rightarrow\ g' \not\to g \text{ 于 } H_\sigma, \quad \forall k\\
\sum_{x\in A_k^{\mathrm{L1}}(t)} s_x \le 524288, \quad \sum_{x\in A_k^{\mathrm{UB}}(t)} s_x \le 131072, \quad \forall k,t\\
\omega(o) = \pi(\sigma(o)), \quad \Delta D = \Delta D_{\mathrm{cut}}(\omega) + \Delta D_{\mathrm{spill}}\\
\Delta D_{\mathrm{cut}} = \Delta D_{\mathrm{DDR}} + \Delta D_{\mathrm{L2}}\\
M_N^{\mathrm{C}} \le M_{N-1}^{\mathrm{C}} \le \cdots \le M_1^{\mathrm{C}}
\end{cases}
\end{aligned}
\tag{19}
$$

其中 ΔD_{L2} 由式 (18) 给出。L2 按逻辑张量编号缓存、容量 1048576 字节、先进先出淘汰，这是评估程序的执行机制，不是对决策变量的约束，它通过 M_N^{C} 体现。求解时用于排序的估价为

$$
\hat{M}_{\mathrm{C}} = \max_k L_k + \max\Big(\frac{D_0 + \Delta D_{\mathrm{DDR}}}{B},\ \frac{\Delta D_{\mathrm{L2}}}{B_{\mathrm{L2}}}\Big) + \delta. \tag{20}
$$

式 (18) 假定可命中的重复读全部命中，是命中的上限，实际命中还取决于读取的时间先后，因此 $\hat{M}_{\mathrm{C}}$ 只用于排序，最终以评估程序为准。

8.4 模型的求解

构造与搜索框架与问题二相同，估价换成式 (20)。估价的变化使求解器不必为躲开重复读而牺牲配平：在无 L2 时，一块被多个核心共用的输入会被算作多份 DDR 流量，装箱倾向于把共用者放在同一核心；有 L2 以后这部分流量转到 L2 池，装箱可以更多地追求负载均衡。候选池中另外加入问题二爬山得到的方案，它的 Task 结构与问题三相同。爬山的评估次数取问题二的一半，因为 L2 的影响较小，爬山的边际收益也较小。

1 核结果按题目要求在只读 Cache 配置下以整图一个子图评估得到，无 L2 配置的结果即问题二的结果。

8.5 求解结果与检验

8.5.1 求解结果

问题三的 400 组方案全部合法。两种配置的对比见表 7 与图 18，逐用例的 makespan 与 Cache 命中率见附录表 A3，总额外数据搬运量见附录表 A4，完整结果见支撑材料中的 result_p3.csv。

表 7　问题三两种配置的对比

核数 N	1	2	3	4	5
无 L2 平均加速比	1.000	1.817	2.605	3.284	3.845
只读 Cache 平均加速比	1.009	1.833	2.637	3.332	3.913
只读 Cache 相对无 L2 的加速比 $\bar{\rho}_N$	1.0086	1.0091	1.0148	1.0202	1.0237
平均 Cache 命中率	0.092	0.155	0.214	0.206	0.254
只读 Cache 平均额外搬运量/MB	12.42	12.75	13.49	13.88	14.37

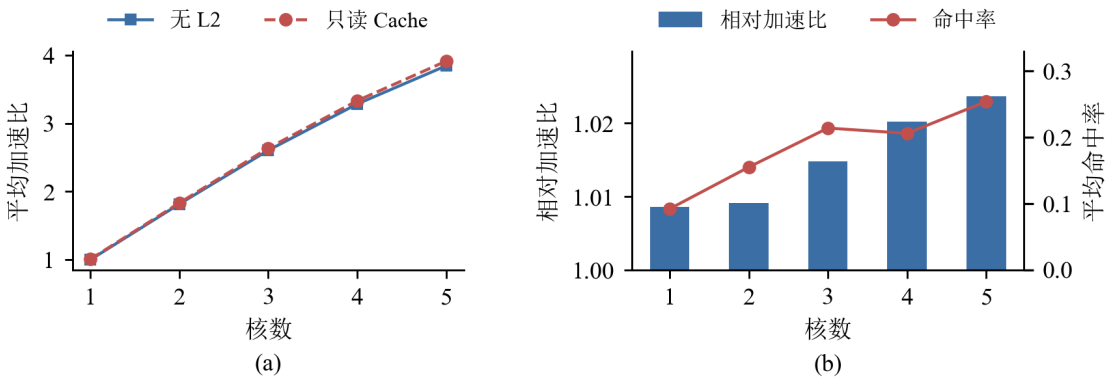


图 18　无 L2 与只读 Cache 的对比：(a) 平均加速比；(b) 相对加速比与命中率

从图 18(a) 可以看出，只读 Cache 的平均加速比在每个核数下都略高于无 L2，两条曲线几乎重合。图 18(b) 给出了相对加速比与命中率，$\bar{\rho}_N$ 从 1 核的 1.0086 增加到 5 核的 1.0237，平均命中率从 0.092 增加到 0.254，其中 4 核的 0.206 略低于 3 核的 0.214。两者总体随核数上升，原因是核数越多，被多个核心共用的输入与广播出去的中间结果越多，可命中的重复读也越多，同时 DDR 通道越拥挤，省下的排队时间越能转化为 makespan 的下降。

命中率在一到三成之间，makespan 却只降了 0.9% 到 2.4%，这与机理分析一致。一方面，DDR 搬运时间只占 makespan 的一部分，而且大部分与计算重叠，省下的搬运只有落在关键路径上的那部分才转化为 makespan 的下降；另一方面，命中率被同时读取的请求拉低，几个核心在同一时刻开工读取同一块输入时，第一次填充之前的请求全部未命中。

综上，只读 Cache 配置下 1 到 5 核的平均加速比分别为 1.009、1.833、2.637、3.332、3.913，相同核数下相对无 L2 的平均加速比分别为 1.0086、1.0091、1.0148、1.0202、1.0237。

8.5.2　多算法对比

与前两问相同，在 80 张中小规模图上比较五种方法在只读 Cache 配置下的 5 核结果，见表 8。

表 8　问题三五种方法的对比，80 张图，$N = 5$

指标	官方示例	表调度	估价择优	实测择优	**本文方法**
平均加速比	1.216	1.577	3.200	3.868	3.922
与最优的平均差距	229.2%	195.0%	37.0%	1.7%	0.0%
命中最优比例	0%	5%	20%	39%	100%
平均用时/s	0.089	0.036	0.665	19.5	113.5

从表 8 可以看出，各方法的排序与前两问一致。官方示例方案与表调度的平均加速比为 1.216 与 1.577，比无 L2 时略高，因为它们切口多、重复读多，L2 能替它们挡掉一部分重复读；估价择优为 3.200，实测择优为 3.868，本文方法为 3.922。实测择优与估价择优的平均差距为 1.7% 与 37.0%，本文方法每个用例约需 113 秒。L2 的存在没有改变各环节的作用，最终仍采用实测择优加爬山。

8.5.3 L2 容量与带宽的影响

在 80 张中小规模图上固定 5 核下问题三的最终方案，把 L2 容量取 256 KB 到 4 MB 五档、带宽取 60 到 1000 字节每周期五档，逐一由评估程序重评，结果如图 19 所示，(a) 为容量的影响，(b) 为带宽的影响，纵轴为同一方案无 L2 与有 L2 的 makespan 之比的平均值。

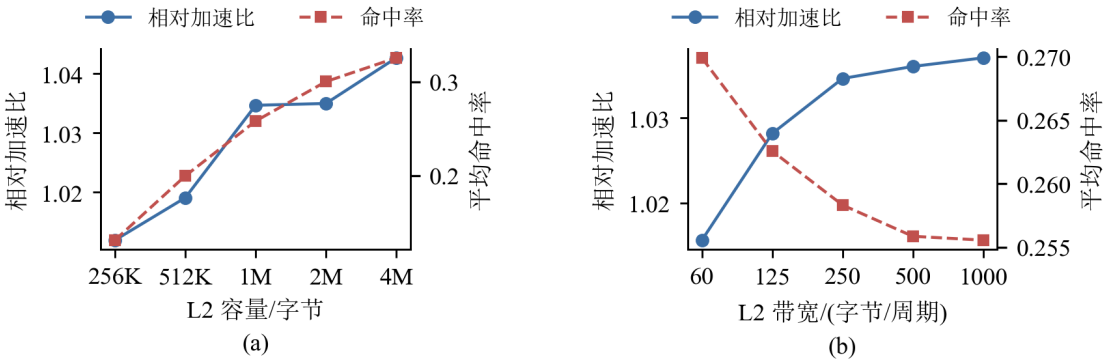


图 19 L2 参数对 5 核结果的影响，80 张图：(a) 容量；(b) 带宽

从图 19(a) 可以看出，容量从 256 KB 增加到 4 MB，平均命中率由 0.131 升到 0.326，相对加速比由 1.0119 升到 1.0427，两者随容量单调上升。容量的作用在于能放下更大的张量、在两次读取之间留得更久。256 KB 时大块输入放不进去，相对加速比 1.0119 只有 1 MB 时的三分之一左右。从 1 MB 加到 2 MB 相对加速比只由 1.0347 变为 1.0350，从 2 MB 加到 4 MB 却升到 1.0427，而命中率在这两段分别升了 0.04 与 0.03，增幅相近。可见 2 MB 到 4 MB 这一段多命中的读取更多落在关键路径上，容量的收益不只取决于多命中了多少字节，还取决于命中的是哪一次读取。图 19(b) 中带宽从 60 增加到 1000 字节每周期，相对加速比由 1.0156 变为 1.0371。带宽为 60 时 L2 与 DDR 一样慢，命中只起到分流的作用；带宽超过 250 以后命中的数据很快读完，再加带宽几乎没有收益。带宽升高时平均命中率由 0.270 小幅降到 0.256，因为 L2 读得越快，各核后续读取发出得越早，更多读取落在缓存填充完成之前，这与同时读取无法命中的机理一致，但读取更快的收益大于这点命中损失。两个参数相比，容量的影响略大于带宽，但都远小于核数与 DDR 带宽的影响。

这一结果对切图也有指导意义。L2 越大、越快，重复读的代价越低，切图时越可以放手让共用输入的算子分到不同核心，以换取更好的负载均衡；反之则应让共用者尽量留在同一核心。在题目给定的 1 MB 与 250 字节每周期下，只读 Cache 带来的平均收益最多为 5 核时的 2.4%，所以问题三的最优方案与问题二高度相似，89.8% 的用例组合在两问中采用了同一套方案。

8.5.4 模型的检验

采用条件弱化的方法检验。把 L2 容量压到 256 KB 时，能进入 L2 的张量大幅减少，模型应当向无 L2 的问题二退化：此时同一方案有无 L2 的 makespan 之比为 1.0119，与 1 只差 1.19%，而 1 MB 时为 1.0347，容量越小越接近无 L2，退化方向与模型一致。另一方面，问题三的每组方案都与问题二的方案做过互换复核，99.2% 的用例组合满足 $M_N^{\mathrm{C}} \le M_N^{\mathrm{noL2}}$。其余 3 个组合中有 L2 反而略慢，最多慢 0.02%，原因是命中的读取改走 L2 通道后各核读取的先后发生了变化。可见加入 L2 几乎不会使结果变差，这与只读 Cache 只分流、不增加搬运的机理一致。

8.6 本章小结

本章在问题二的基础上把搬运拆成 DDR 与 L2 两个带宽池，建立了**双带宽池的共享 L2 多核切图调度模型**。只读 Cache 配置下 1 到 5 核的平均加速比为 1.009 到 3.913，相对无 L2 的加速比为 1.0086 到 1.0237，平均命中率为 0.092 到 0.254。L2 能命中一到三成的读取，但 makespan 只降 0.9% 到 2.4%，原因在于 DDR 搬运大部分与计算重叠，且同时发出的读取无法命中；容量的影响略大于带宽，带宽超过 250 字节每周期以后几乎没有收益。

九、 模型的灵敏度分析

三问的共同瓶颈是共享的 DDR 带宽。本章先看 DDR 搬运时间在 makespan 中的占比如何随核数变化，再把 DDR 总带宽 B 取 30、45、60、90、120 字节每周期五档，在 80 张中小规模图上固定问题二的 5 核最终方案，单核基线与多核方案都在同一带宽下重评，考察加速比对 B 的灵敏度。

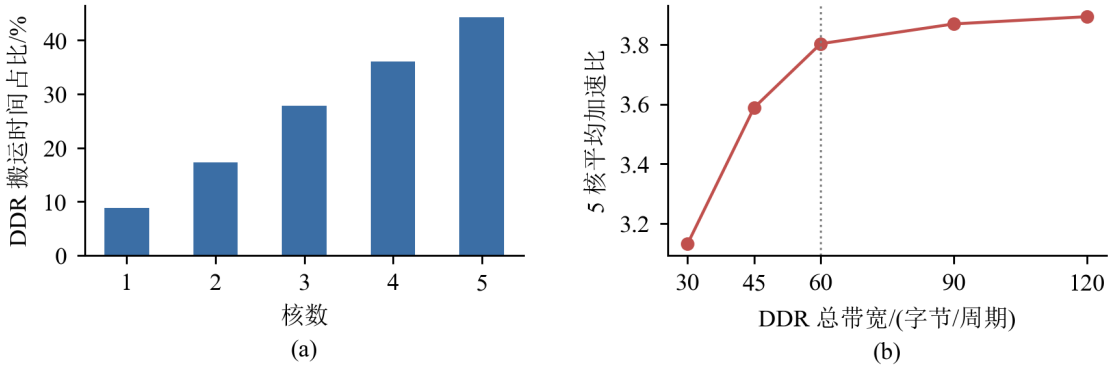


图 20 DDR 带宽的影响：(a) 搬运时间占比随核数的变化；(b) 5 核加速比随 DDR 带宽的变化

如图 20(a) 所示，按 $(D_0 + \Delta D)/B$ 与 makespan 之比计算，问题二中 DDR 搬运时间的平均占比从 1 核的 8.8% 升到 5 核的 44.3%。计算时间随核数摊薄，而原图搬运 D_0 不变、额外搬运 ΔD 还随切口增加，所以核数越多，makespan 中由带宽决定的部分越大。这正是平均加速比曲线在 4、5 核处弯下来的原因。

图 20(b) 给出了 5 核平均加速比随 B 的变化。带宽从 60 降到 30 时，平均加速比由 3.803 降到 3.131，下降 17.7%；带宽升到 120 时，平均加速比为 3.894，上升 2.4%。带宽降低时加速比下降得更快，说明在题目给定的 60 字节每周期附近，多核方案已经处在带宽受限的边缘，带宽再少一些就会成为主要瓶颈，而带宽加倍带来的收益较为有限，因为此时瓶颈又回到计算负载的配平上。可见加速比对 DDR 带宽较为敏感，DDR 带宽是多核 NPU 设计中需要与核数一同权衡的关键参数；对于切图算法而言，带宽越紧，越应该把减少切口放在负载均衡之前。

L2 容量与带宽的影响已在问题三中给出，在题目给定的参数附近，两者对 makespan 的影响都在 5% 以内，属于不敏感的参数。

十、 模型的评价、改进与推广

10.1 模型的优点

第一，三问的模型一脉相承。三个模型共用同一套决策变量与合法性约束，逐问只改变 Task 的归属单位、等待项与带宽池，求解框架也完全复用，三问的方案还能互相作为起点并做互换复核，结果之间可以相互印证。

第二，方案的合法性由构造保证。核心亲和拓扑序区间切分给出的方案天然满足无环与次序相容两条硬约束，并有严格证明，搜索过程中不需要任何修复，1200 组方案全部一次通过官方评估程序的检查。

第三，估价与实测分工明确。闭式估价把 makespan 拆成计算负载、共享带宽上的搬运与等待三项，毫秒级即可给大量候选排序；实测驱动的爬山弥补了估价看不见换出换入的缺陷。两者结合，在五种方法的对比中取得了最高的平均加速比。

第四，结果具有单调性与可复现性。补空核心的方案保证加速比随核数不降，随机种子由用例、核数与问题号定死，任何一组结果都可以重新生成。

10.2 模型的缺点

第一，估价把相互重叠的计算与搬运直接相加，对计算与搬运高度重叠的图会偏保守，排序时可能错过个别好的候选，只能靠实测择优来补。

第二，核内算子超过 9000 个的 20 张大图单次评估耗时较长，没有做爬山，只取两轮实测择优中较好的方案，这部分用例仍有改进空间。

10.3 模型的改进

若能在估价中近似计入换出换入，例如按核内张量的存活区间估计缓存峰值，估价与实测的一致性会更高，实测的次数可以大幅减少，大图也就能做局部搜索。另外，问题三目前只在估价层面把重复读分给 L2，若在同核次序中有意把共用同一输入的 Task 在时间上错开，可以提高命中率，使 L2 的作用更充分。

10.4 模型的推广

第一，本文的模型可以直接用于其他多核加速器上的算子切分与映射，例如多芯粒的 AI 芯片与多 GPU 的流水线并行，只需把带宽池与等待项换成对应平台的参数。

第二，聚簇、装箱加实测择优的框架适用于目标函数只能靠模拟得到的一类调度问题，例如片上网络的任务映射与云上工作流调度，闭式估价负责快速排序，模拟负责最终把关。

参考文献

[1] Liao H, Tu J, Xia J, et al，Ascend: a scalable and unified architecture for ubiquitous deep neural network computing，2021 IEEE International Symposium on High-Performance Computer Architecture，789-801，2021.

[2] Chen T, Moreau T, Jiang Z, et al，TVM: an automated end-to-end optimizing compiler for deep learning，13th USENIX Symposium on Operating Systems Design and Implementation，578-594，2018.

[3] 中国研究生创新实践系列大赛组委会，“华为杯”第二十三届中国研究生数学建模竞赛 A 题：通用神经网络处理器下的多核调度问题及附件，https://cpipc.acge.org.cn/，2026-09-24.

[4] Kwok Y K, Ahmad I，Static scheduling algorithms for allocating directed task graphs to multiprocessors，ACM Computing Surveys，31(4)：406-471，1999.

[5] Ahn B H, Lee J, Lin J M, et al，Ordering chaos: memory-aware scheduling of irregularly wired neural networks for edge devices，Proceedings of Machine Learning and Systems，2：44-57，2020.

[6] Herrmann J, Özkaya M Y, Uçar B, et al，Multilevel algorithms for acyclic partitioning of directed acyclic graphs，SIAM Journal on Scientific Computing，41(4)：A2117-A2145，2019.

[7] Yang T, Gerasoulis A，DSC: scheduling parallel tasks on an unbounded number of processors，IEEE Transactions on Parallel and Distributed Systems，5(9)：951-967，1994.

[8] Graham R L，Bounds on multiprocessing timing anomalies，SIAM Journal on Applied Mathematics，17(2)：416-429，1969.

[9] Topcuoglu H, Hariri S, Wu M Y，Performance-effective and low-complexity task scheduling for heterogeneous computing，IEEE Transactions on Parallel and Distributed Systems，13(3)：260-274，2002.

[10] Kernighan B W, Lin S，An efficient heuristic procedure for partitioning graphs，The Bell System Technical Journal，49(2)：291-307，1970.

[11] Belady L A，A study of replacement algorithms for a virtual-storage computer，IBM Systems Journal，5(2)：78-101，1966.

附录

附录 A　支撑材料清单

```
支撑材料/
  结果/
    result_p1.csv        问题一逐用例 2~5 核结果
    result_p2.csv        问题二，列同上
    result_p3.csv        问题三，另含 Cache 命中率与 1 核结果
    experiments.csv      多算法对比、L2 参数与 DDR 带宽实验的原始结果
  AI工具使用说明.md      智能工具辅助编写代码的说明
  方案.zip               1200 个方案文件 case_xxx_nN_pP.json
  代码/
    main.py              命令行入口：读图、求解、输出方案并调用评估
    graph.py             计算图读取与结构统计
    partition.py         链式与锥形聚簇、装箱、局部改进
    solve.py             亲和拓扑序区间切分与候选生成
    score.py             额外搬运、等待与 makespan 估价
    fasteval.py          进程内调用官方评估函数
    final_run.py         实测择优与爬山
    cross.py             三问方案互换复核与单调补齐
    experiments.py       多算法对比与参数实验
    features.py          数据特征统计
```

附录 B　逐用例结果

表 A1　问题一逐用例的 makespan 与总额外数据搬运量，单位为周期与字节

用例	M_1	M_2	ΔD_2	M_3	ΔD_3	M_4	ΔD_4	M_5	ΔD_5
001	233110	116868	1152	78545	2304	58984	3456	47502	4608
002	261945	133236	98304	102693	242688	96441	276480	76292	184320
003	602212	602212	0	439572	2445144	354969	2851446	260392	3468180
004	392357	199074	110592	134043	221184	104188	331776	88341	442368
005	95618	95618	0	70541	861060	58897	1028642	58897	1028642
006	82837	41780	43400	31153	30080	30413	220594	27435	229832
007	1012562	507494	27648	338094	55296	254398	82944	203834	110592
008	487605	244266	0	163359	0	123060	0	100603	0
009	153261	92198	0	92198	0	92198	0	87238	400596
010	86969	43614	20736	30076	72960	29856	58752	29856	86400
011	184697	92646	180224	64488	1245184	46880	540672	40858	1728512
012	59088	29744	4032	20682	72964	15738	27648	13475	58002
013	263949	132198	8192	88351	16384	66982	24576	54363	32768
014	17698626	8897730	71757240	5980663	77770920	4521815	81886104	3681573	87046152
015	177362	89066	230180	58958	285068	46044	440856	38455	342536
016	7715523	7715523	0	6877573	119561440	6877573	119561440	6877573	119561440
017	162810	81359	87552	56546	257474	43729	259200	38706	339840
018	1004367	500374	2896896	337216	4036608	253652	4558848	209242	5369860
019	81144	41826	27176	27078	54336	21462	82468	18346	100420
020	532746	266580	0	177950	0	133704	0	107650	0
021	6567937	3194570	3174400	2138267	7675904	1550324	4608000	1250168	11431936
022	56780	28333	6912	19102	13824	14472	20736	11828	13824
023	122614	60654	73728	40412	74016	30319	281280	24301	280992
024	2555343	2555343	0	2253443	39329192	2236654	39329036	2202377	39328822
025	4863026	2431840	27648	1622164	55296	1217444	82944	975282	110592
026	99203	48242	20496	41655	21648	40034	202800	40034	202800
027	2363952	1120962	1744896	763738	4792320	559196	3317760	467276	6359040

用例	M_1	M_2	ΔD_2	M_3	ΔD_3	M_4	ΔD_4	M_5	ΔD_5
028	43836040	23638628	277473600	16770711	277369056	13608312	277760640	11491035	277818912
029	37555	19740	0	12743	0	11028	0	9383	0
030	2808034	1399149	6877184	933256	8949248	698838	7811072	560819	10660864
031	1414369	706317	5622272	472905	4800000	369209	7469568	289307	8209920
032	41443	21519	18436	14156	12432	10022	6288	9996	6288
033	1774196	891746	6301224	589266	6812736	443810	7213664	356830	7948954
034	842996	417089	1543168	278227	2734080	210002	3392000	169942	3773440
035	169571	95406	0	95406	0	95406	0	92549	681776
036	931510	466068	1152	311345	2304	233584	3456	187182	4608
037	210001	105374	204800	73969	679936	53320	614400	43800	1925120
038	1364555	680290	6387712	460001	7944192	341821	9039360	279506	8357888
039	1435262	708147	9773056	484201	13705216	373553	14647296	349662	15433728
040	253454	124680	24576	109567	36228	109567	36228	109567	36228
041	6003957	2983392	30750720	1989512	34489344	1497746	34397184	1203359	38209536
042	1686740	833723	6718752	558880	7496992	422274	8777280	352612	9959424
043	677656	353177	385024	268777	233480	268777	233480	268777	233480
044	154407	106836	3467104	106798	3467104	106714	3466848	106714	3466848
045	125614	63059	0	42601	0	31878	0	26164	0
046	276455	166578	3521632	142176	4021280	124612	4513952	124612	4513952
047	326508	326508	0	259452	1435588	223884	1623858	176914	1750624
048	222220	220907	25192	220788	25258	220519	25324	220445	25324
049	196276	166270	0	140250	1008688	102960	658002	90760	876696
050	149690	127179	60820	105670	662040	88124	769224	81017	716984
051	607628	597017	393264	479544	9045242	401212	10290476	362774	10028404
052	180301	90487	102912	60913	152064	46464	291840	39407	328704
053	1472686	774466	5959552	546106	5882944	492232	6360992	492232	6360992
054	815051	692292	2241218	557486	1663268	442174	2242118	406503	2690600
055	118835	60179	57856	37147	180784	30566	201770	24642	168960
056	277285	274903	0	197577	826248	160139	1015646	151122	1314684
057	59726	33550	288	21741	288	16223	52288	14028	14112
058	4539717	2271316	33431552	1554831	30400512	1154519	42622976	1004819	32104448
059	829473	369318	360448	261117	3338240	185216	1081344	150736	2207744
060	888789	444880	2603042	317137	3522824	234185	3641656	224348	3693880
061	77191	38254	34560	25565	40704	21234	95488	17803	163340
062	2854809	1457390	79872	974551	199680	974551	199680	974551	199680
063	1070397	542594	165888	420717	248832	417297	319488	416097	347136
064	19116	19116	0	19116	0	17569	109954	17569	109954
065	53068	26729	13968	17938	27936	13737	41760	11366	55728
066	359070	345512	20784	242535	2028290	167771	1404280	159646	1339746
067	63834834	33768704	264024576	23568859	263599744	18473155	263174912	15927184	262750080
068	362095	342266	67260	301013	2382678	253153	2551958	223526	2433354
069	25144	24879	59816	18409	202866	18220	195664	15145	294466
070	117300	59885	145936	38552	259072	29722	347144	29722	347144
071	18919	18716	52776	14895	161864	12262	212652	12262	212652
072	23897828	11833111	113945376	7940908	112945952	5993260	122669856	4794501	127574704
073	10081626	5980423	141121024	4515227	140671616	3903787	140222208	3517515	139772800
074	1682239	817173	6860800	555069	7102464	412790	8269824	339281	8228872
075	1441809	1441809	0	1102360	3584260	898574	4425778	861677	4659200
076	6185113	3069852	31259648	2057399	32532480	1534268	36393984	1235462	37289984
077	495320	246479	57344	164192	114688	130314	172032	118573	172032
078	2296143	1260358	12535344	934776	13263312	690924	13146912	649952	13717680
079	31052251	15446486	31780864	10181692	39960576	7635624	30445568	6075976	47677440
080	292138	145856	1921024	100325	2793472	90584	1650688	90584	1650688
081	151795	76860	0	51943	0	39588	0	30943	0
082	721041	721041	0	480847	1332784	391435	1600800	391435	1600800
083	1057993	644672	14241216	510989	14812032	450029	14977600	435764	15114112
084	2507412	1255052	0	836220	0	629028	0	503472	0
085	3187899	3187899	0	2202443	5881592	1738075	6345314	1398547	7399436
086	118403	118403	0	85040	879344	70366	1041600	65724	1181172
087	3974071	1985234	9999360	1384726	9953280	1069661	11773440	885864	10990080
088	243623	204253	54440	187944	1604222	153623	1716748	135918	1664536
089	693092	346832	238720	243614	477440	178160	716160	147432	954880
090	1178363	769224	21498480	579795	20896776	565326	21240408	509864	22257168
091	9186769	4577377	57023968	3062743	62303552	2314405	66813600	1879886	70317568
092	4815554	2828252	66389088	2231804	66784736	1927728	67180384	1661234	66980256
093	74205	37827	8192	25103	16384	19779	24576	15876	32768

用例	M_1	M_2	ΔD_2	M_3	ΔD_3	M_4	ΔD_4	M_5	ΔD_5
094	94892	47640	106272	32057	156960	23944	332128	19413	325472
095	2084853	1042890	0	695775	0	524422	0	420852	0
096	84194	42233	67392	28326	76608	26984	66240	26720	134556
097	11491509	5568273	14602240	3575902	17145856	2626908	14221312	2101771	20992000
098	177430	89756	0	89756	0	89756	0	77504	766188
099	1622999	806668	7233536	537413	8822784	405898	9334784	336011	10326016
100	151223	76740	76800	57811	72192	57811	72192	57811	72192

表 A2　问题二逐用例的 makespan 与总额外数据搬运量，单位为周期与字节

用例	M_1	M_2	ΔD_2	M_3	ΔD_3	M_4	ΔD_4	M_5	ΔD_5
001	233110	116868	1152	78545	2304	58984	3456	47502	4608
002	261945	132292	49152	89456	76800	72618	98304	60997	104448
003	602212	549292	840326	386701	1729676	307618	2619026	240170	3459224
004	392357	198970	125952	134043	221184	104136	347136	87177	562176
005	95618	86172	294438	60978	659564	51812	949362	46392	1249656
006	82837	41780	43400	27382	100112	27244	122008	27131	95648
007	1012562	507494	27648	338094	55296	254398	82944	203834	110592
008	487605	244266	0	163359	0	123060	0	100603	0
009	153261	92198	0	92198	0	79803	276848	70629	415506
010	86969	43614	20736	29830	59136	23411	157088	22595	190884
011	184697	92646	180224	62556	1196032	46880	540672	39086	1681408
012	59088	29744	4032	19606	75468	15324	88944	13046	99248
013	263949	132198	8192	88351	16384	66947	28672	54227	36864
014	17698626	8897730	71757240	5980663	77770920	4521815	81886104	3681573	87046152
015	177362	88787	254752	58850	279884	45852	492608	38455	342536
016	7715523	7704938	393252	3209083	18280	3209083	18280	3209083	18280
017	162810	81359	87552	54271	223768	40011	300576	33239	524342
018	1004367	499275	2919680	337216	4036608	253192	4589056	209114	5521408
019	81144	41206	34084	27078	54336	20711	64804	18165	88132
020	532746	266580	0	177950	0	133704	0	107650	0
021	6567937	3194570	3174400	2138267	7675904	1550324	4608000	1250168	11431936
022	56780	28333	6912	19102	13824	14472	20736	11728	15884
023	122614	60654	73728	39728	283052	30319	281280	24058	297380
024	2555343	1383028	101128	1055092	5860	1053983	5832	1031425	1643172
025	4863026	2431840	27648	1622164	55296	1217444	82944	975282	110592
026	99203	48096	109196	34546	196412	34546	196412	33793	311120
027	2363952	1120962	1744896	752497	4444160	559196	3317760	453794	6170624
028	43836040	23638628	277473600	16770711	277369056	13608312	277760640	11491035	277818912
029	37555	19636	4104	12743	0	10287	47160	8772	38970
030	2808034	1399149	6877184	933256	8949248	698838	7811072	560819	10660864
031	1414369	708882	5714432	471700	4532736	364927	7474464	289307	8209920
032	41443	20533	34844	14156	12432	9863	34988	9733	37036
033	1774196	887742	6339108	588543	6814272	443702	7236704	358569	8101480
034	842996	417089	1543168	278479	2723840	209399	3336704	169961	3764224
035	169571	95406	0	95174	10852	95148	10916	73753	379122
036	931510	466068	1152	311345	2304	233584	3456	187182	4608
037	210001	105374	204800	72574	1239040	53320	614400	43800	1925120
038	1364555	678730	6282752	458041	7909888	341821	9039360	279506	8428544
039	1435262	708277	9469952	473680	14372864	373553	14647296	349662	15433728
040	253454	124475	16384	109197	192	104228	209702	99672	413036
041	6003957	2983392	30750720	1989512	34489344	1497746	34397184	1203359	38209536
042	1686740	833723	6718752	556589	7616064	422274	8777280	344828	10562560
043	677656	353177	385024	268777	233480	268777	233480	268777	233480
044	154407	73465	1803488	71255	1736416	71143	1736672	70966	1736416
045	125614	63059	0	42601	0	31878	0	26164	0
046	276455	166578	3521632	142176	4021280	124612	4513952	124612	4513952
047	326508	310832	444230	229247	867852	197370	1271122	154148	1684568
048	222220	196661	265030	150861	599180	121195	617008	112758	645972
049	196276	164255	414950	117557	345260	89767	467250	78361	605240
050	149690	119821	382114	88400	376404	67835	472130	62900	564680
051	607628	328061	66200	229096	33716	229096	33716	228082	132020
052	180301	90487	102912	60913	152064	46139	264192	37386	420894

用例	M_1	M_2	ΔD_2	M_3	ΔD_3	M_4	ΔD_4	M_5	ΔD_5
053	1472686	774466	5959552	546106	5882944	492232	6360992	492232	6360992
054	815051	640582	582278	508744	1203084	393435	1625042	345318	1961944
055	118835	53306	126996	35992	96360	29261	194612	24515	144384
056	277285	245423	524726	176951	539972	140644	758970	133731	928536
057	59726	30162	58896	21689	57376	15911	110400	14028	14112
058	4539717	2263810	33923072	1554831	30400512	1154519	42622976	1004819	32104448
059	829473	369318	360448	257601	3227648	185216	1081344	150736	2207744
060	888789	441161	2614408	317127	3504392	234063	3604792	227797	3626296
061	77191	38254	34560	25565	40704	21019	80648	17525	117520
062	2854809	1431677	1658880	972995	138240	859374	417792	752464	718848
063	1070397	542416	58368	420539	107520	322422	168960	291215	319488
064	19116	17218	33670	14330	87244	11859	108786	10230	116918
065	53068	26729	13968	17938	27936	13697	47912	11214	79280
066	359070	291016	673276	191735	1377410	152932	1178844	142067	1167012
067	63834834	33768704	264024576	23568859	263599744	18473155	263174912	15927184	262750080
068	362095	331019	841696	255352	2067564	216520	2316132	189369	2323810
069	25144	19911	138372	14281	273612	14225	292142	12268	318162
070	117300	55484	186690	38552	259072	29619	347140	25200	442392
071	18919	15029	92260	12923	220234	9599	251052	9573	248558
072	23897828	11833111	113945376	7940908	112945952	5993260	122669856	4794501	127574704
073	10081626	5980423	141121024	4515227	140671616	3903787	140222208	3517515	139772800
074	1682239	817173	6860800	554419	7057408	412790	8269824	338803	8564744
075	1441809	1302029	1290374	997283	2681996	790948	3886482	723683	4603928
076	6185113	3069852	31259648	2057399	32532480	1534268	36393984	1235462	37289984
077	495320	246479	57344	164192	114688	130314	172032	118573	172032
078	2296143	1235519	12639576	881603	12905184	657515	13484880	590890	13418304
079	31052251	15446486	31780864	10181692	39960576	7635624	30445568	6075976	47677440
080	292138	145856	1921024	100325	2793472	90515	1675264	89919	1712128
081	151795	76836	2052	51855	2052	38946	47160	30943	0
082	721041	641189	584454	416524	1112972	354689	1544466	296888	2092664
083	1057993	644672	14241216	510989	14812032	450029	14977600	435764	15114112
084	2507412	1255052	0	836220	0	629028	0	503472	0
085	3187899	2975395	1979270	2033355	3792524	1590697	5532050	1274709	7197848
086	118403	102241	355494	74468	678636	61506	954738	54793	1353432
087	3974071	1985234	9999360	1384726	9953280	1069661	11773440	885864	10990080
088	243623	204166	1344	149732	1501962	126546	1605456	112089	1528340
089	693092	346832	238720	241666	549120	178160	716160	147432	954880
090	1178363	690990	9837600	541586	20805000	541389	20792712	442640	12003648
091	9186769	4577377	57023968	3062743	62303552	2314405	66813600	1879886	70317568
092	4815554	2828252	66389088	2231804	66784736	1927728	67180384	1649430	66980256
093	74205	37691	12288	25103	16384	19643	28672	15841	36864
094	94892	47640	106272	31972	205120	23549	322400	19413	321888
095	2084853	1042890	0	695775	0	524422	0	420852	0
096	84194	42159	83796	28081	89344	22878	161788	22878	161788
097	11491509	5574860	12398592	3563487	17043456	2626908	14221312	2101771	20992000
098	177430	89756	0	79625	421996	67994	516240	67022	467088
099	1622999	805900	7217152	537477	8756224	405866	9338880	335731	10317824
100	151223	75845	73728	53124	160334	52457	191342	51549	209266

表 A3　问题三只读 Cache 配置逐用例的 makespan 与命中率，makespan 单位为周期，无 L2 配置的结果同表 A2

用例	M_1	M_2	M_3	M_4	M_5	h_1	h_2	h_3	h_4	h_5
001	233110	116868	78545	58984	47502	0.000	0.000	0.000	0.000	0.000
002	261945	132292	89456	72618	60997	0.000	0.000	0.000	0.000	0.000
003	602212	549292	386700	305917	237168	0.000	0.159	0.284	0.367	0.441
004	392357	198970	134043	104136	87177	0.000	0.000	0.000	0.000	0.013
005	95618	86172	60820	51060	45367	0.000	0.176	0.310	0.327	0.398
006	82837	41780	27382	27244	27131	0.029	0.074	0.133	0.146	0.147
007	1012562	507494	338094	254398	203834	0.000	0.000	0.000	0.000	0.000
008	487605	244266	163359	123060	100603	0.000	0.000	0.000	0.000	0.000
009	153261	92198	92198	79796	70591	0.000	0.000	0.000	0.084	0.189
010	86969	43614	29830	23411	22595	0.000	0.051	0.114	0.239	0.216

用例	M_1	M_2	M_3	M_4	M_5	h_1	h_2	h_3	h_4	h_5
011	184697	92646	62556	46880	38557	0.000	0.000	0.563	0.000	0.592
012	59088	29744	19606	15324	12739	0.000	0.008	0.086	0.103	0.152
013	263949	132198	88351	66947	54227	0.000	0.000	0.000	0.000	0.000
014	17689602	8888616	5942569	4500714	3672408	0.123	0.228	0.289	0.274	0.270
015	177362	88787	58850	45643	38447	0.169	0.213	0.239	0.254	0.199
016	7715523	7704936	3209083	3209083	3209083	0.000	0.500	0.002	0.002	0.002
017	162810	81359	54271	40011	33200	0.040	0.141	0.257	0.331	0.340
018	1003110	498831	337216	252000	209114	0.344	0.518	0.544	0.627	0.611
019	81144	41206	27078	20711	18165	0.000	0.133	0.127	0.117	0.103
020	532746	266580	177950	133704	107650	0.000	0.000	0.000	0.000	0.000
021	6558837	3193634	2136157	1550324	1249380	0.501	0.218	0.541	0.000	0.570
022	56780	28333	19102	14472	11728	0.000	0.031	0.061	0.089	0.069
023	122614	60654	39728	30319	24058	0.000	0.145	0.328	0.187	0.162
024	2555343	1383026	1055092	1053983	1028260	0.000	0.199	0.001	0.001	0.753
025	4863026	2431840	1622164	1217444	975282	0.000	0.000	0.000	0.000	0.000
026	99203	48096	34535	34472	33793	0.000	0.180	0.243	0.243	0.243
027	2359515	1120962	752497	559196	453794	0.269	0.089	0.415	0.000	0.205
028	43836040	23615145	16770711	13585418	11450794	0.000	0.005	0.000	0.003	0.011
029	37555	19636	12743	10094	8547	0.000	0.016	0.000	0.132	0.178
030	2807891	1398912	932283	698674	560766	0.260	0.415	0.521	0.529	0.552
031	1414159	707569	470887	364329	289307	0.261	0.386	0.403	0.464	0.550
032	41443	20533	14156	9863	9650	0.000	0.047	0.043	0.111	0.105
033	1774080	886977	588041	442826	356483	0.096	0.274	0.296	0.334	0.374
034	842733	416957	278228	208831	169223	0.309	0.395	0.531	0.573	0.580
035	169571	95406	95174	95148	73717	0.000	0.000	0.000	0.000	0.162
036	931510	466068	311345	233584	187182	0.000	0.000	0.000	0.000	0.000
037	210001	105374	72574	53320	43013	0.000	0.000	0.553	0.000	0.642
038	1363312	678611	457141	341821	277097	0.174	0.227	0.346	0.359	0.384
039	1435262	705031	470658	365012	297920	0.000	0.220	0.425	0.508	0.487
040	253454	124449	109197	104228	99670	0.000	0.013	0.000	0.049	0.105
041	6001575	2981318	1988508	1494017	1201975	0.102	0.171	0.234	0.241	0.261
042	1675633	832308	555428	421887	343912	0.319	0.404	0.391	0.464	0.335
043	677656	353177	268777	268777	268777	0.033	0.212	0.140	0.140	0.140
044	107814	62248	47000	46978	46939	0.820	0.592	0.729	0.729	0.729
045	125614	63059	42601	31878	26164	0.000	0.000	0.000	0.000	0.000
046	244994	137552	121494	105932	105932	0.753	0.574	0.431	0.313	0.313
047	326508	310832	229301	197309	153960	0.000	0.166	0.264	0.350	0.423
048	222220	196661	150727	121077	112719	0.000	0.023	0.191	0.181	0.196
049	196276	164254	117531	89747	78288	0.000	0.046	0.136	0.164	0.234
050	149690	119819	88355	67750	62834	0.000	0.135	0.303	0.320	0.318
051	607628	328061	229096	229096	228082	0.000	0.143	0.077	0.077	0.250
052	180301	90487	60913	46139	37328	0.000	0.113	0.158	0.238	0.300
053	1428109	768103	546106	492232	492232	0.625	0.462	0.385	0.285	0.285
054	815051	640582	508743	393434	345312	0.000	0.068	0.164	0.177	0.187
055	118835	53192	35992	29249	24094	0.100	0.101	0.112	0.145	0.140
056	277285	245284	176938	140631	133409	0.000	0.147	0.245	0.164	0.319
057	59726	30160	21688	15792	14028	0.219	0.110	0.065	0.207	0.040
058	4539717	2262281	1554831	1142629	1004819	0.000	0.113	0.000	0.269	0.000
059	815938	369318	255476	185216	150736	0.610	0.000	0.684	0.000	0.542
060	887801	440903	316918	233818	227441	0.423	0.475	0.408	0.430	0.430
061	77191	38248	25507	21019	17393	0.000	0.060	0.070	0.107	0.126
062	2854809	1431677	972995	859374	752464	0.000	0.000	0.000	0.000	0.000
063	1070397	542416	420539	322422	291215	0.000	0.000	0.000	0.000	0.000
064	19116	17218	14301	11794	10206	0.000	0.020	0.278	0.356	0.323
065	53068	26729	17938	13697	11212	0.000	0.045	0.057	0.058	0.150
066	359070	291016	191475	152919	141484	0.000	0.181	0.312	0.328	0.305
067	63834834	33768704	23568859	18473155	15927184	0.000	0.000	0.000	0.000	0.000
068	362095	331019	255219	215130	188101	0.000	0.147	0.257	0.304	0.194
069	25144	19911	14070	14018	11884	0.000	0.278	0.333	0.344	0.337
070	117300	55484	38552	29619	24631	0.000	0.245	0.282	0.260	0.292
071	18919	15010	12828	9394	9331	0.000	0.111	0.347	0.242	0.224
072	23890756	11819724	7936337	5973202	4782436	0.026	0.087	0.088	0.190	0.217
073	10081626	5980423	4515227	3903787	3517515	0.000	0.000	0.000	0.000	0.000
074	1678120	815846	549404	409885	335960	0.167	0.219	0.285	0.397	0.286
075	1441809	1302029	997277	790865	723148	0.000	0.183	0.278	0.304	0.324
076	6183098	3065951	2052990	1532122	1229983	0.134	0.282	0.344	0.377	0.389

用例	M_1	M_2	M_3	M_4	M_5	h_1	h_2	h_3	h_4	h_5
077	495320	246479	164192	130314	118573	0.000	0.026	0.025	0.025	0.014
078	2296143	1201498	868404	650373	521355	0.000	0.279	0.244	0.245	0.526
079	31052251	15446486	10175315	7635624	6075472	0.000	0.000	0.137	0.000	0.244
080	291826	145475	99290	83168	77128	0.169	0.393	0.436	0.553	0.435
081	151795	76836	51855	38891	30943	0.000	0.002	0.002	0.043	0.000
082	721041	641189	416480	354689	296473	0.000	0.158	0.209	0.205	0.340
083	946145	535329	401109	333151	310731	0.796	0.708	0.672	0.643	0.676
084	2507412	1255052	836220	629028	503472	0.000	0.000	0.000	0.000	0.000
085	3187899	2975395	2033354	1590697	1274709	0.000	0.154	0.228	0.299	0.349
086	118403	102238	74284	60801	53419	0.000	0.169	0.226	0.319	0.314
087	3969779	1984074	1382680	1069661	885864	0.183	0.268	0.247	0.313	0.274
088	243623	204166	149247	124922	110084	0.000	0.000	0.315	0.257	0.184
089	693092	346832	241666	178160	147432	0.000	0.000	0.000	0.000	0.000
090	1178363	659564	467288	400061	371427	0.000	0.385	0.400	0.420	0.700
091	9168820	4565812	3046566	2306119	1872762	0.105	0.196	0.245	0.199	0.332
092	4333631	2358868	1792534	1470508	1351088	0.794	0.759	0.654	0.648	0.581
093	74205	37691	25103	19643	15841	0.000	0.000	0.000	0.000	0.000
094	94892	47640	31972	23538	19331	0.000	0.096	0.214	0.277	0.307
095	2084853	1042890	695775	524422	420852	0.000	0.000	0.000	0.000	0.000
096	84194	42159	28081	22878	22878	0.000	0.236	0.238	0.353	0.353
097	11491509	5575035	3563824	2626908	2101417	0.000	0.137	0.215	0.042	0.293
098	177430	89756	79607	67994	66974	0.000	0.000	0.256	0.299	0.293
099	1618902	804656	535772	402775	325703	0.252	0.388	0.486	0.535	0.501
100	151223	75845	53124	52457	51549	0.000	0.142	0.269	0.295	0.313

表 A4　问题三只读 Cache 配置逐用例的总额外数据搬运量，单位为字节

用例	ΔD_1	ΔD_2	ΔD_3	ΔD_4	ΔD_5
001	0	1152	2304	3456	4608
002	0	49152	76800	98304	104448
003	0	840326	1729676	2619026	3459224
004	0	125952	221184	347136	562176
005	0	294438	659564	949362	1249656
006	16400	43400	100112	122008	95648
007	0	27648	55296	82944	110592
008	0	0	0	0	0
009	0	0	0	276848	415506
010	0	20736	59136	157088	190884
011	0	180224	1196032	540672	1681408
012	0	4032	75468	88944	105388
013	0	8192	16384	28672	36864
014	71010720	71757240	77770920	81886104	87046152
015	180256	254752	279884	492608	342536
016	0	393252	18280	18280	18280
017	22528	87552	223768	300576	524342
018	1970176	2919680	4036608	4589056	5521408
019	0	34084	54336	64804	88132
020	0	0	0	0	0
021	4501504	3174400	7675904	4608000	11431936
022	0	6912	13824	20736	15884
023	0	73728	283052	281280	297380
024	0	101128	5860	5832	1643172
025	0	27648	55296	82944	110592
026	0	109196	196412	196412	311120
027	1400832	1744896	4444160	3317760	6170624
028	277412256	277473600	277369056	277760640	277818912
029	0	4104	0	53312	63582
030	5519360	6877184	8949248	7811072	10660864
031	3491328	5714432	4529668	7474464	8209920
032	0	34844	12432	34988	37036
033	4592640	6339108	6814272	7236704	8101480
034	1079832	1543168	2735616	3343880	3764224
035	0	0	10852	10916	379122

用例	ΔD_1	ΔD_2	ΔD_3	ΔD_4	ΔD_5
036	0	1152	2304	3456	4608
037	0	204800	1239040	614400	1925120
038	5627904	6282752	7909888	9039360	8626688
039	7929856	9469952	14426112	17752064	16060416
040	0	16384	192	209702	413036
041	30898176	30750720	34489344	34397184	38209536
042	6629376	6718752	7616064	8777280	10562560
043	49160	385024	233480	233480	233480
044	4438112	1769824	2979744	2979744	2979744
045	0	0	0	0	0
046	3176288	3521632	4021280	4513952	4513952
047	0	444230	867852	1271122	1684568
048	0	265030	599180	617008	645972
049	0	414950	345260	467250	605240
050	0	382114	376404	472130	564680
051	0	66200	33716	33716	132020
052	0	102912	152064	264192	420894
053	5405536	5959552	5882944	6360992	6360992
054	0	582278	1203084	1625042	1961944
055	69632	139292	96360	194612	160304
056	0	524726	539972	758970	928536
057	94216	58896	57376	122696	14112
058	28696576	33923072	30400512	42622976	32104448
059	1691648	360448	3237888	1081344	2207744
060	2068520	2614408	3504392	3604792	3626296
061	0	34560	40704	80648	117512
062	0	1658880	138240	417792	718848
063	0	58368	107520	168960	319488
064	0	33670	87244	108786	116918
065	0	13968	27936	47912	79280
066	0	673276	1377410	1178844	1167012
067	264449408	264024576	263599744	263174912	262750080
068	0	841696	2067564	2316132	2323810
069	0	138372	273612	292142	318162
070	0	186690	259072	347140	448560
071	0	92260	220044	251052	237228
072	108613680	113945376	112945952	121950256	127574704
073	141570432	141121024	140671616	140222208	139772800
074	6070280	6860800	7057408	8269824	8523784
075	0	1290374	2681996	3886482	4603928
076	28280832	31259648	32532480	36393984	37289984
077	0	57344	114688	172032	172032
078	12644568	12762456	13000848	13484880	13573440
079	32448512	31780864	39960576	30445568	47677440
080	548864	1921024	2818048	3985408	2887680
081	0	2052	2052	47160	0
082	0	584454	1112972	1544466	2092664
083	13934688	14241216	14812032	14977600	15058048
084	0	0	0	0	0
085	0	1979270	3792524	5532050	7197848
086	0	355494	678636	954738	1353432
087	7902720	9999360	9953280	11773440	10990080
088	0	1344	1501962	1605456	1528340
089	0	238720	549120	716160	954880
090	21233592	9837600	21016344	21535416	11918400
091	53947008	57023968	62303552	66813600	70317568
092	66000096	66389088	66784736	67180384	66980256
093	0	12288	16384	28672	36864
094	0	106272	205120	322400	321888
095	0	0	0	0	0
096	0	83796	89344	161788	161788
097	10584064	12398592	17043456	14221312	20992000
098	0	0	421996	516240	467088
099	5484544	7221248	8749056	9334784	10317824
100	0	73728	160334	191342	209266

附录 C　核心代码

以下代码为核心亲和拓扑序区间切分，对应第六章的式 (11)，三问共用。

```
def affinity_order(g, core, finish, ncore):
    indeg = {v: len(g.pred[v]) for v in g.eligible}
    ready = [[] for _ in range(ncore)]
    for v in g.eligible:
        if not indeg[v]:
            heapq.heappush(ready[core[v]], (finish[v], v))
    out, cur, left = [], 0, len(g.eligible)
    while left:
        if not ready[cur]:
            cands = [(ready[k][0][0], k) for k in range(ncore) if ready[k]]
            if not cands:
                raise RuntimeError('affinity order stalled')
            cur = min(cands)[1]
        _, v = heapq.heappop(ready[cur])
        out.append(v)
        left -= 1
        for w in g.succ[v]:
            indeg[w] -= 1
            if not indeg[w]:
                heapq.heappush(ready[core[w]], (finish[w], w))
    return out


def _no_cross_core(g, core):
    """ 核心之间一条数据边都没有。

    场景 A 里一核只出一个子图看着最省边界，但只要核间有一条边，后一个 Task 就
    得等前一个整个跑完再加 1000 周期，两个核心直接串成一条线，比单核还慢。所以
    这种落法只在完全没有跨核边时才用。
    """
    for v in g.eligible:
        kv = core[v]
        for w in g.succ[v]:
            if core[w] != kv:
                return False
    return True


def plan_from_cores(g, core, ncore, one_per_core):
    """ 算子到核心的分配落成方案。"""
    if one_per_core and _no_cross_core(g, core):
        used = sorted({core[v] for v in g.eligible})
        sid = {k: i for i, k in enumerate(used)}
        sched = [[] for _ in range(ncore)]
        for k, i in sid.items():
            sched[k].append(i)
        return ({'node_to_subgraph': {str(v): sid[core[v]]
                                      for v in g.eligible},
                 'core_schedules': sched}, len(used))
    finish = _est_finish(g, core, ncore)
    order = affinity_order(g, core, finish, ncore)
    mapping, sched = {}, [[] for _ in range(ncore)]
    sg, last = -1, None
    for v in order:
        k = core[v]
        if k != last:
            sg += 1
            sched[k].append(sg)
            last = k
        mapping[str(v)] = sg
    return {'node_to_subgraph': mapping, 'core_schedules': sched}, sg + 1
```

以下代码按 Task 归属计算额外搬运量，对应式 (2) 与式 (18)，场景 A 传入子图，场

景 B 传入核心。

```
def added_bytes(g, owner, l2=False):
    """owner 把算子映射到它所属的 Task，场景 A 传子图、场景 B 传核心。

    l2 为真时区分两个带宽池：多个 Task 重复读同一块 DDR 输入，第一次未命中走
    DDR，其余几次从只读 Cache 上命中，走的是与 DDR 无关的那条带宽。所以这部分
    不该记进 DDR 流量，切图也就不必为了躲重复读而牺牲配平。
    """
    el = set(g.eligible)
    readers, writers = g._ddr_users if hasattr(g, '_ddr_users') else \
        ddr_users(g)
    g._ddr_users = (readers, writers)
    add = 0
    cache = 0
    for tid, t in g.tensors.items():
        sz = t['size']
        if not sz:
            continue
        if t['pos'] == 'DDR':
            # 原图只搬入一次，几个 Task 用它就要各读一遍
            rs = readers.get(tid)
            if rs:
                n = len({owner[v] for v in rs})
                extra = sz * (n - 1)
                if l2 and sz <= L2_CAP:
                    cache += extra
                else:
                    add += extra
            continue
        prod = [u for u in g.producers.get(tid, ()) if u in el]
        if not prod:
            continue
        cons = [v for v in g.consumers.get(tid, ()) if v in el]
        home = owner[prod[0]]
        outside = {owner[v] for v in cons} - {home}
        if outside:
            # 写出去一次，外面每个 Task 各读一遍
            if l2 and sz <= L2_CAP and len(outside) > 1:
                # 边界搬运插的 COPY_IN 同样会查 Cache，广播出去的第二份起能命中
                add += sz * 2
                cache += sz * (len(outside) - 1)
            else:
                add += sz * (1 + len(outside))
    return (add, cache) if l2 else add
```